

MedTrack: AWS Cloud-Enabled Healthcare Management System

Project Description:

In today's fast-evolving healthcare landscape, efficient communication and coordination between doctors and patients are crucial. MedTrack is a cloud-based healthcare management system that streamlines patient doctor interactions by providing a centralized platform for booking appointments, managing medical histories, and enabling diagnosis submissions. To address these challenges, the project utilizes Flask for backend development, AWS EC2 for hosting, and DynamoDB for managing data. MedTrack allows patients to register, log in, book appointments, and submit diagnosis reports online. The system ensures real-time notifications, enhancing communication between doctors and patients regarding appointments and medical submissions. Additionally, AWS Identity and Access Management (IAM) is employed to ensure secure access control to AWS resources, allowing only authorized users to access sensitive data. This cloud-based solution improves accessibility and efficiency in healthcare services for all users.

Scenario 1: Efficient Appointment Booking System for Patients

In the MedTrack system, AWS EC2 provides a reliable infrastructure to manage multiple patients accessing the platform simultaneously. For example, a patient can log in, navigate to the appointment booking page, and easily submit a request for an appointment. Flask handles backend operations, efficiently retrieving and processing user data in real-time. The cloud-based architecture allows the platform to handle a high volume of appointment requests during peak periods, ensuring smooth operation without delays.

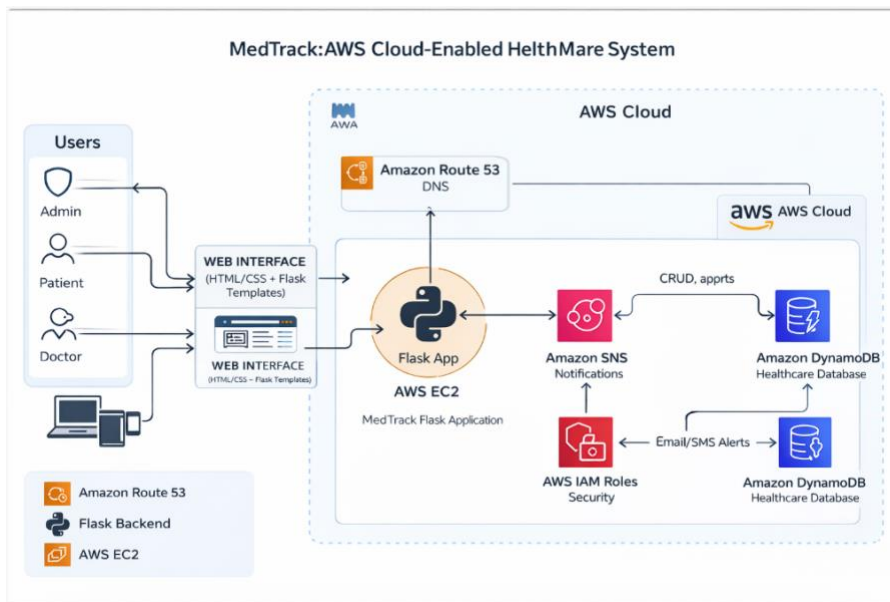
Scenario 2: Secure User Management with IAM

MedTrack utilizes AWS IAM to manage user permissions and ensure secure access to the system. For instance, when a new patient registers, an IAM user is created with specific roles and permissions to access only the features relevant to them. Doctors have their own IAM configurations, allowing them access to patient records and appointment details while maintaining strict security protocols. This setup ensures that sensitive data is accessible only to authorized users.

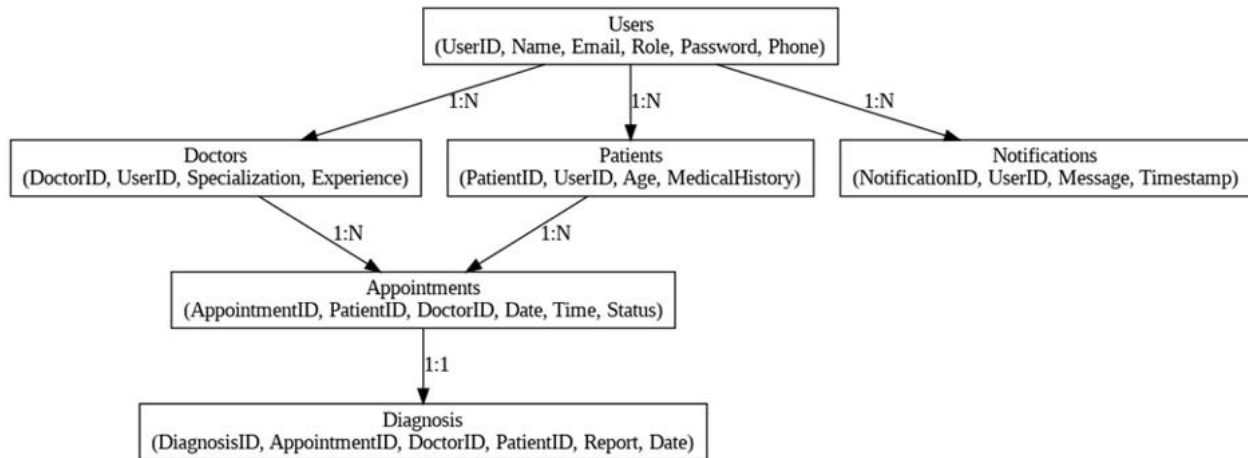
Scenario 3: Easy Access to Medical History and Resources

The MedTrack system provides doctors and patients with easy access to medical histories and relevant resources. For example, a doctor logs in to view a patient's medical history and upcoming appointments. They can quickly access, and update records as needed. Flask manages real-time data fetching from DynamoDB, while EC2 hosting ensures the platform performs seamlessly even when multiple users access it simultaneously, offering a smooth and uninterrupted user experience.

AWS ARCHITECTURE



Entity Relationship (ER) Diagram:



Pre-requisites:

1. **AWS Account Setup:** [aws account setup](#)
2. **AWS IAM (Identity and Access Management):** [IAM Overview](#)
3. **Amazon EC2 Basics:** [EC2 basics](#)
4. **DynamoDB Basics:** [dynamodb](#)
5. **GIT Documentation:** [GIT](#)
6. **VS Code Installation:** [vscode](#)

Project WorkFlow:

Epic 1. Backend Development and Application Setup

Develop the Backend Using Flask.

Integrate AWS Services Using boto3.

Epic 2. AWS Account Setup and Login

Set up an AWS account if not already done.

Log in to the AWS Management Console

Epic 3. DynamoDB Database Creation and Setup

Create a DynamoDB Table.

Configure Attributes for User Data and Book Requests.

Epic 4. SNS Notification Setup

Create SNS topics for book request notifications.

Subscribe users and library staff to SNS email notifications.

Epic 5. IAM Role Setup

Create IAM Role

Attach Policies

Epic 6. EC2 Instance Setup

Launch an EC2 instance to host the Flask application.

Configure security groups for HTTP, and SSH access.

Epic 7. Deployment on EC2

Upload Flask Files0

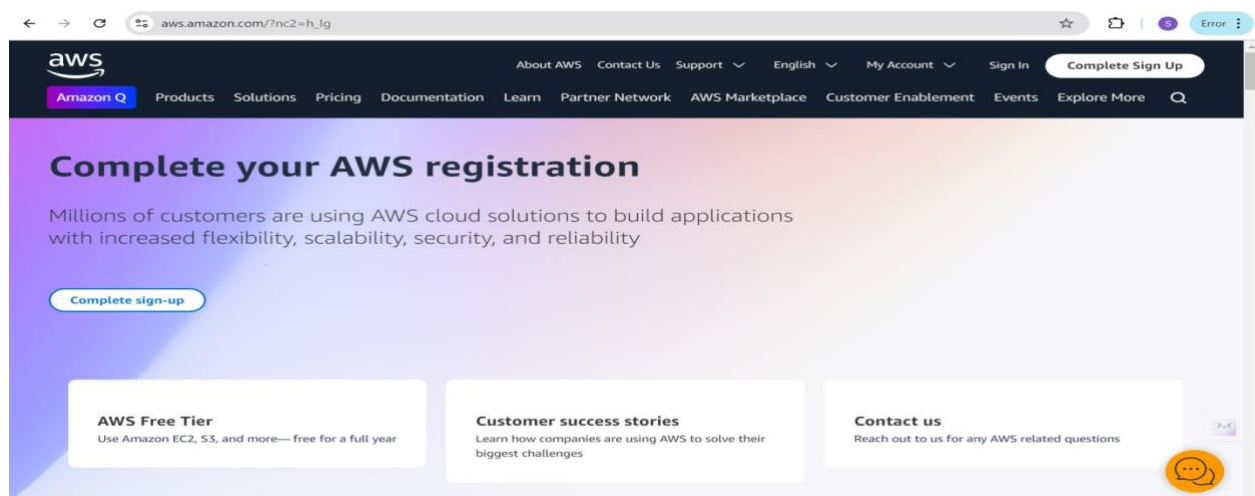
Run the Flask App

Epic 8. Testing and Deployment

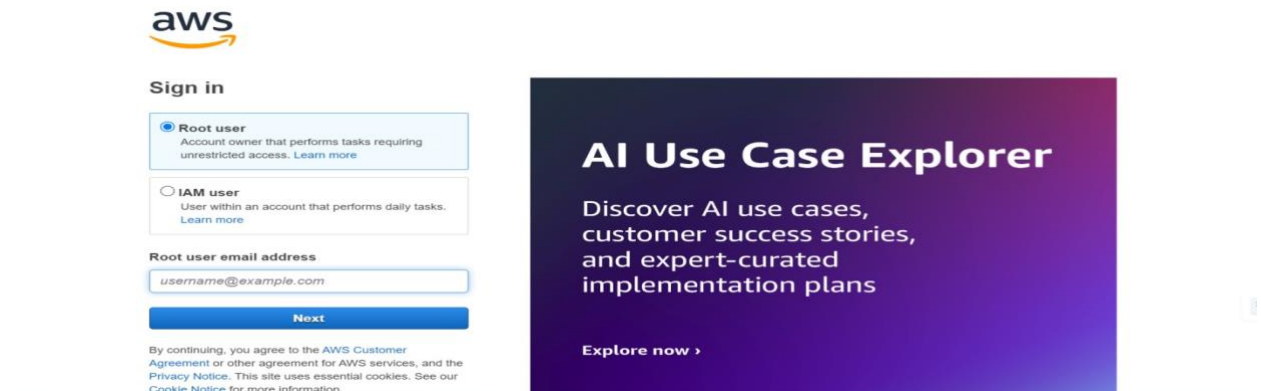
Conduct functional testing to verify user registration, login, book requests, and notifications.

Milestone 1: AWS Account Setup and Login

- **Activity 1.1: Set up an AWS account if not already done.**
 - Sign up for an AWS account and configure billing settings.



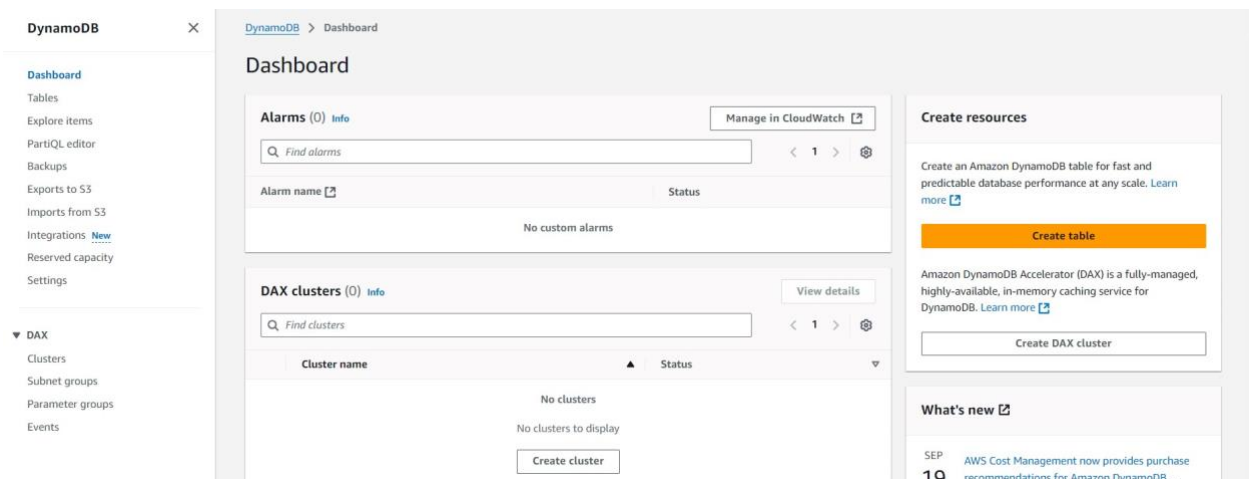
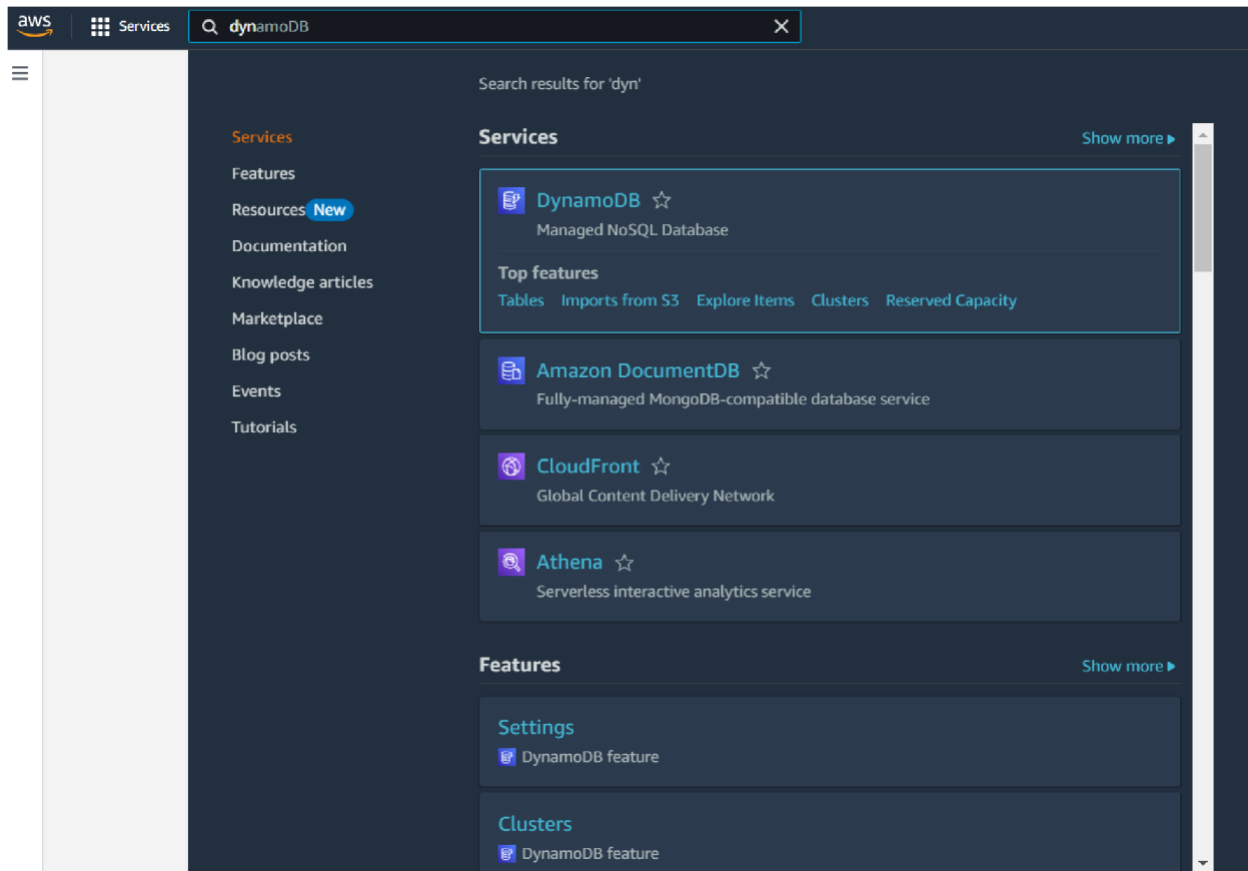
- **Activity 1.2: Log in to the AWS Management Console**
 - After setting up your account, log in to the [AWS Management Console](#).

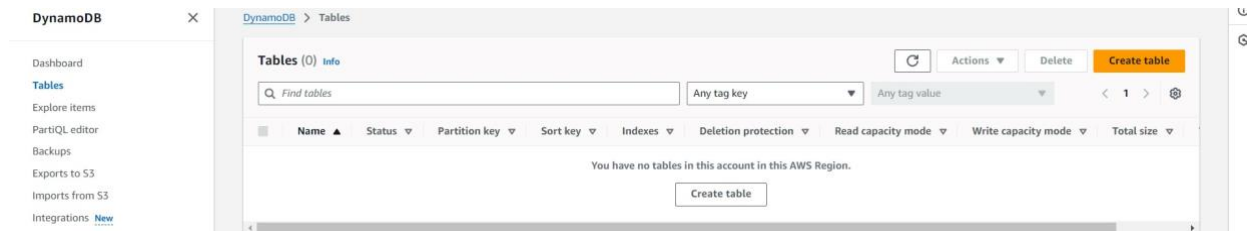


Milestone 2: DynamoDB Database Creation and Setup

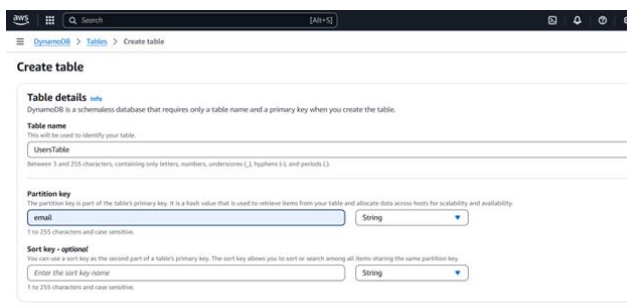
- **Activity 2.1: Navigate to the DynamoDB**

- In the AWS Console, navigate to DynamoDB and click on create tables.





- **Activity 2.2: Create DynamoDB tables for storing registration details and booking records.**
 - Create a travel-Users table for storing registered user details with partition key “Email” with type String and click on create tables.



Create table

Table details

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.
UsersTable

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and place data across hosts for scalability and availability.
email String

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.
Enter the sort key name String

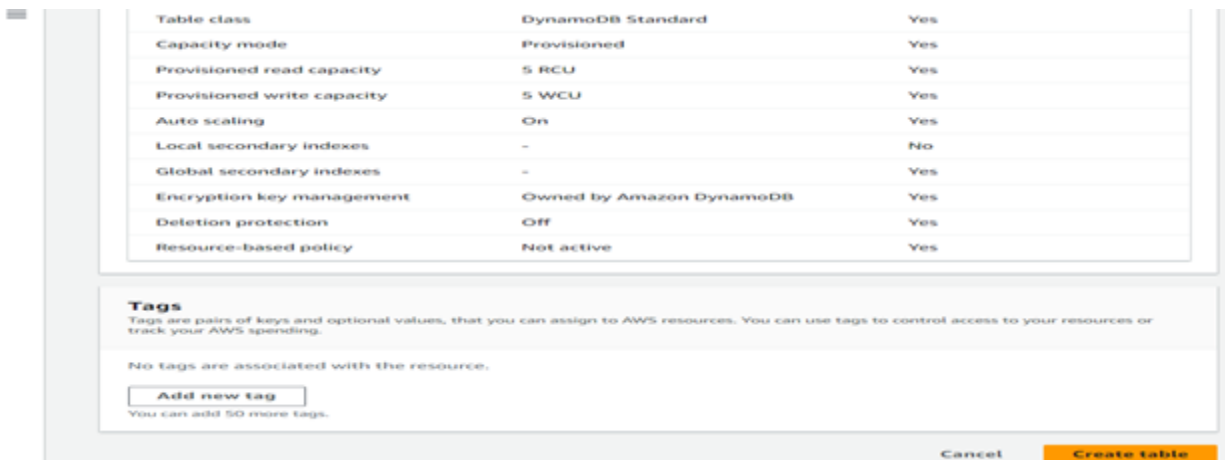


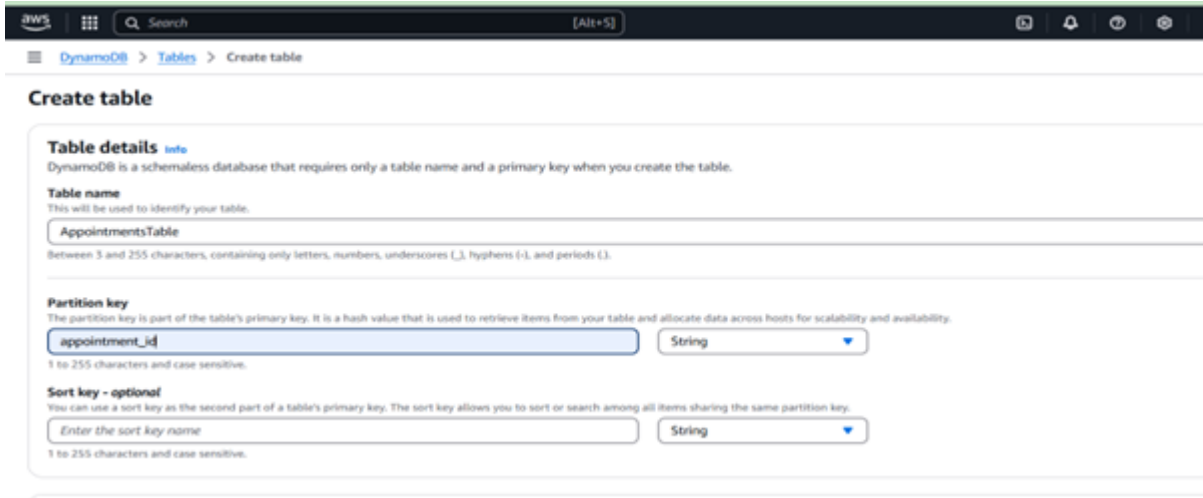
Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

Tags
Tags are pairs of keys and optional values, that you can assign to AWS resources. You can use tags to control access to your resources or track your AWS spending.

No tags are associated with the resource.

Add new tag
You can add 50 more tags.

Cancel Create table



The screenshot shows the AWS Management Console for DynamoDB, specifically the 'Create table' page. The breadcrumb navigation at the top reads 'DynamoDB > Tables > Create table'. The main heading is 'Create table'. Below this, there's a 'Table details' section with an 'info' link. A descriptive paragraph states: 'DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.' The 'Table name' section includes a text input field containing 'AppointmentsTable' and a note: 'Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.)'.

The 'Partition key' section explains that it's part of the table's primary key and is used for retrieval and data allocation. The input field contains 'appointment_id' and a dropdown menu is set to 'String'. A note below states: '1 to 255 characters and case sensitive.'

The 'Sort key - optional' section explains that it's the second part of a primary key for sorting. The input field contains 'Enter the sort key name' and the dropdown menu is set to 'String'. A note below states: '1 to 255 characters and case sensitive.'

- Follow the same steps to create a Bookings for storing booking records with email as the partition key and booking_id as the sort key.

Table class	DynamoDB Standard	Yes
Capacity mode	Provisioned	Yes
Provisioned read capacity	5 RCU	Yes
Provisioned write capacity	5 WCU	Yes
Auto scaling	On	Yes
Local secondary indexes	-	No
Global secondary indexes	-	Yes
Encryption key management	Owned by Amazon DynamoDB	Yes
Deletion protection	Off	Yes
Resource-based policy	Not active	Yes

Tags
 Tags are pairs of keys and optional values, that you can assign to AWS resources. You can use tags to control access to your resources or track your AWS spending.

No tags are associated with the resource.

[Add new tag](#)
 You can add 50 more tags.

[Cancel](#)
[Create table](#)

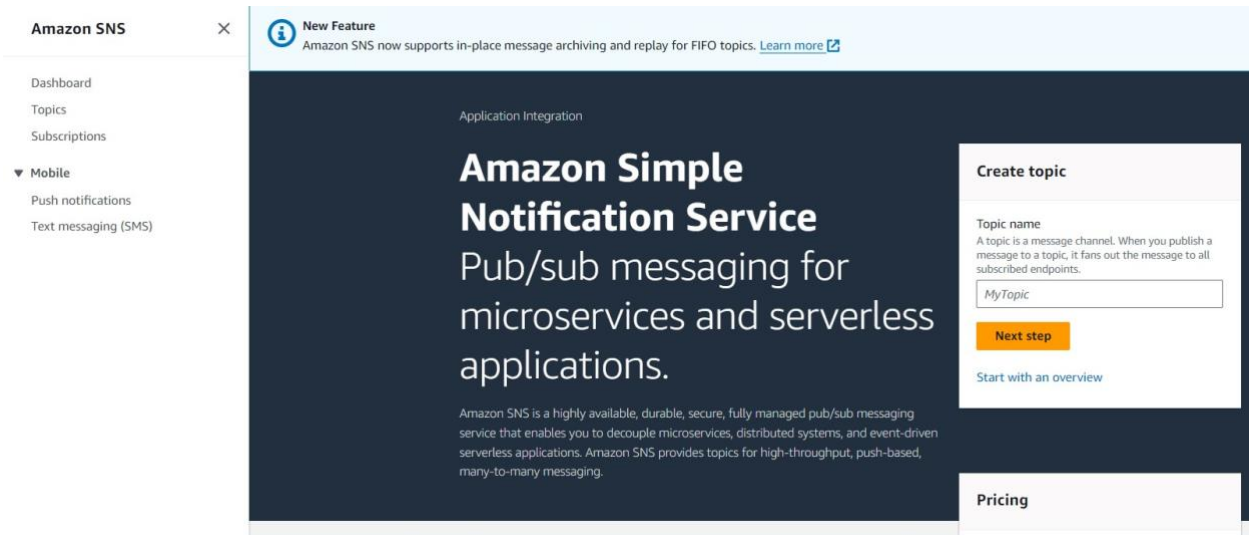
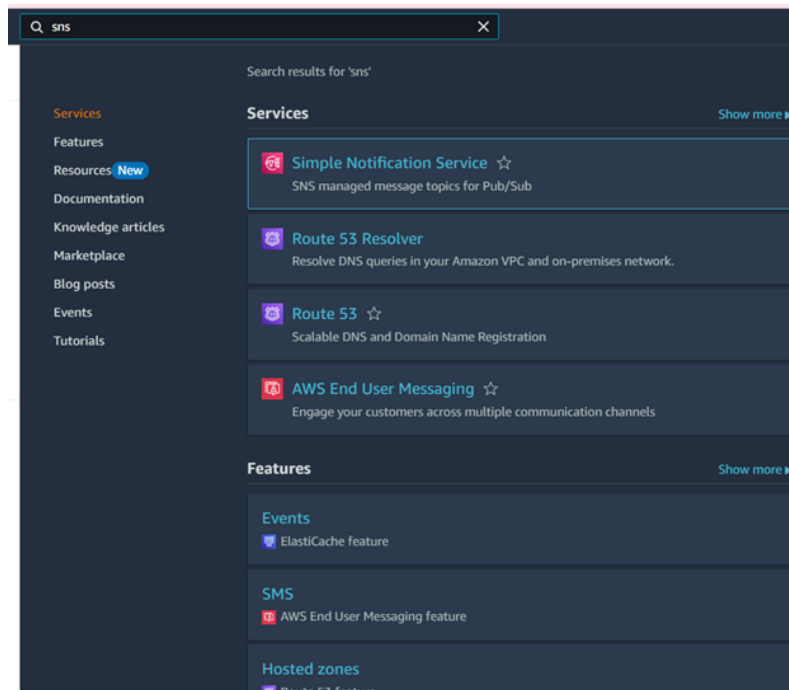
Tables (2/10) [Info](#)

< 1 >

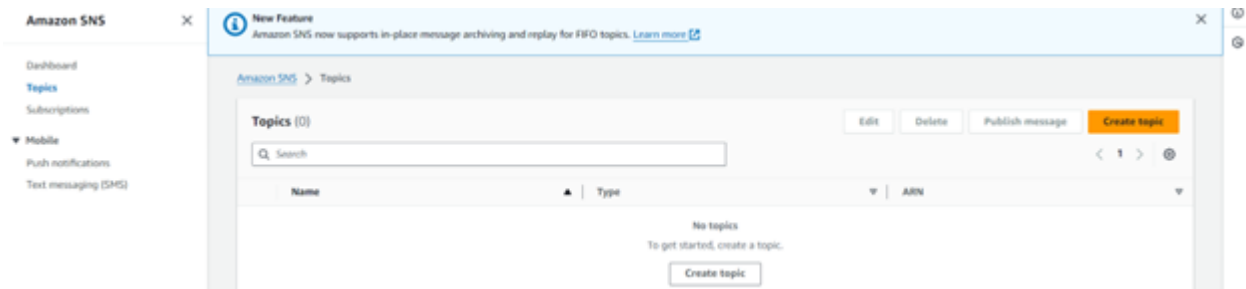
	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Deletion protection
☒	AppointmentsTable	Active	appointment_id (S)	-	0	0	Off
☐	NextGenHospital_Appointments	Active	appointment_id (S)	-	0	0	Off
☐	NextGenHospital_ContactMessages	Active	message_id (S)	-	0	0	Off
☐	NextGenHospital_Doctors	Active	doctor_id (S)	-	0	0	Off
☐	NextGenHospital_PatientRecords	Active	patient_id (S)	-	0	0	Off
☐	NextGenHospital_Users	Active	email (S)	-	0	0	Off
☐	SalonAppointments	Active	appointment_id (S)	user_email (S)	0	0	Off
☐	SalonStylists	Active	stylist_id (S)	-	0	0	Off
☐	SalonUsers	Active	email (S)	-	0	0	Off
☒	UsersTable	Active	email (S)	-	0	0	Off

Milestone 3: SNS Notification Setup

- **Activity 3.1: Create an SNS Topic named BookingConfirmation for sending notifications.**
 - In the AWS Console, search for SNS and navigate to the SNS Dashboard.



- Click on **Create Topic** and choose a name for the topic.



- Choose Standard type for general notification use cases and Click on Create Topic.

Access policy - optional [info](#)
This policy defines who can access your topic. By default, only the topic owner can publish or subscribe to the topic.

Data protection policy - optional [info](#)
This policy defines which sensitive data to monitor and to prevent from being exchanged via your topic.

Delivery policy (HTTP/S) - optional [info](#)
The policy defines how Amazon SNS retries failed deliveries to HTTP/S endpoints. To modify the default settings, expand this section.

Delivery status logging - optional [info](#)
These settings configure the logging of message delivery status to CloudWatch Logs.

Tags - optional
A tag is a metadata label that you can assign to an Amazon SNS topic. Each tag consists of a key and an optional value. You can use tags to search and filter your topics and track your costs. [Learn more](#)

Active tracing - optional [info](#)
Use AWS X-Ray active tracing for this topic to view its traces and service map in Amazon CloudWatch. Additional costs apply.

Cancel [Create topic](#)

[Amazon SNS](#) > [Topics](#) > Create topic

New Feature
Amazon SNS now supports High Throughput FIFO topics. [Learn more](#)

Create topic

Details

Type [info](#)
Topic type cannot be modified after topic is created.

☐ FIFO (first-in, first-out)

- Strictly-preserved message ordering
- Exactly-once message delivery
- Subscription protocols: SQS

☒ Standard

- Best-effort message ordering
- At-least-once message delivery
- Subscription protocols: SQS, Lambda, Data Firehose, HTTP, SMS, email, mobile application endpoints

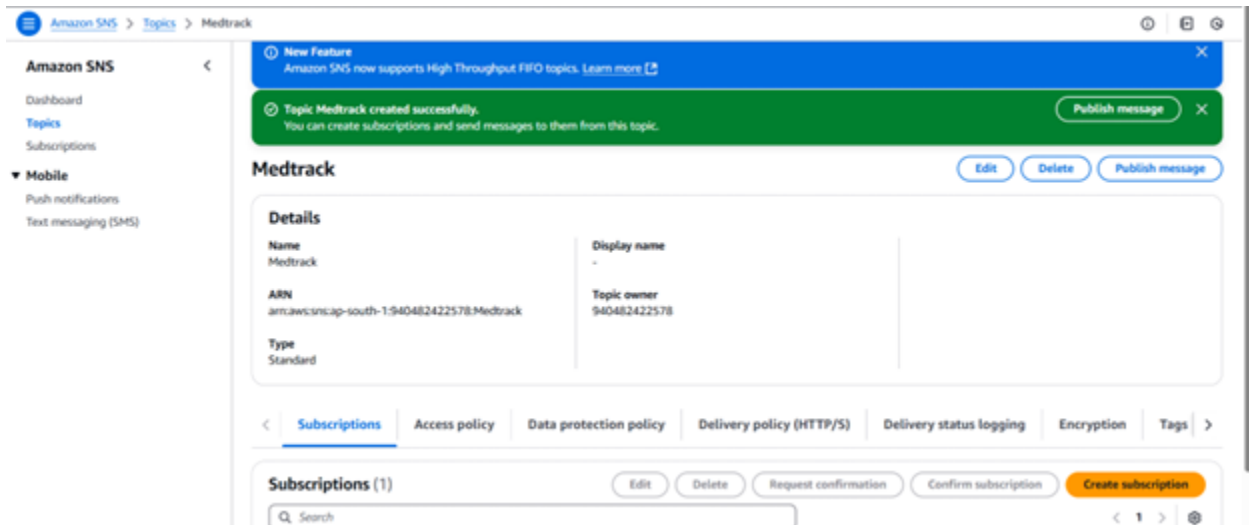
Name

Maximum 256 characters. Can include alphanumeric characters, hyphens (-) and underscores (_).

Display name - optional [info](#)
To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

Maximum 100 characters.

- Configure the SNS topic and note down the **Topic ARN**.



Amazon SNS > Topics > Medtrack

Amazon SNS

- Dashboard
- Topics
- Subscriptions
- ▼ Mobile
 - Push notifications
 - Text messaging (SMS)

Medtrack [Edit] [Delete] [Publish message]

Details

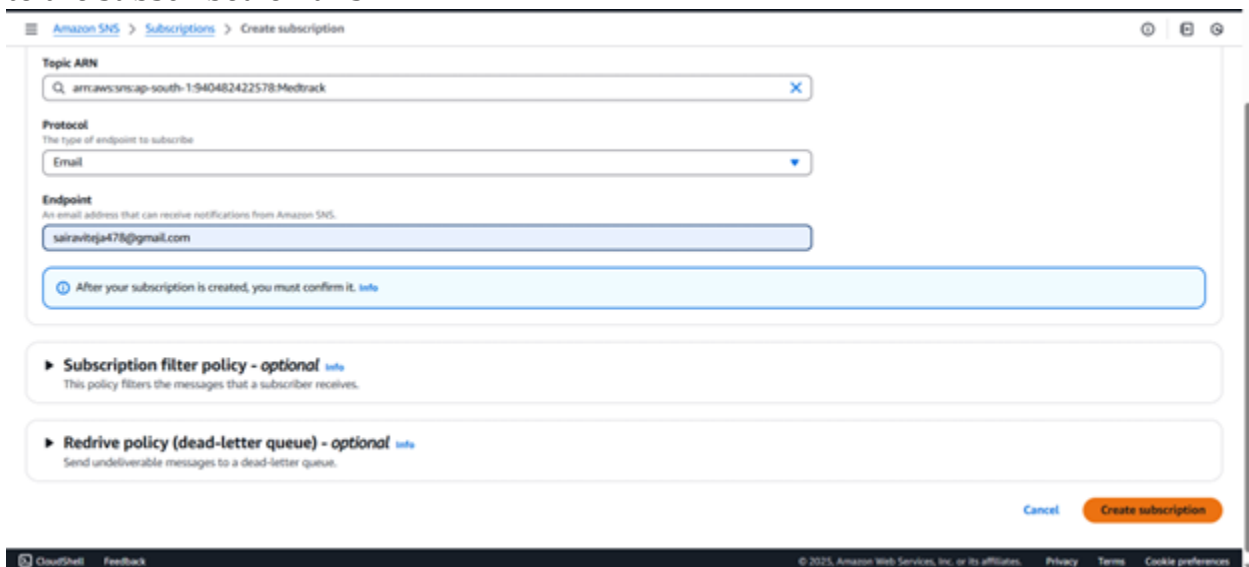
Name Medtrack	Display name -
ARN arn:aws:sns:ap-south-1:940482422578:Medtrack	Topic owner 940482422578
Type Standard	

< **Subscriptions** | Access policy | Data protection policy | Delivery policy (HTTP/S) | Delivery status logging | Encryption | Tags >

Subscriptions (1) [Edit] [Delete] [Request confirmation] [Confirm subscription] [Create subscription]

Search

Activity 3.2: Subscribe users (or admin staff) to this topic via Email. When a book request is made, notifications will be sent to the subscribed emails.



Amazon SNS > Subscriptions > Create subscription

Topic ARN
arn:aws:sns:ap-south-1:940482422578:Medtrack

Protocol
The type of endpoint to subscribe
Email

Endpoint
An email address that can receive notifications from Amazon SNS.
sairaviteja478@gmail.com

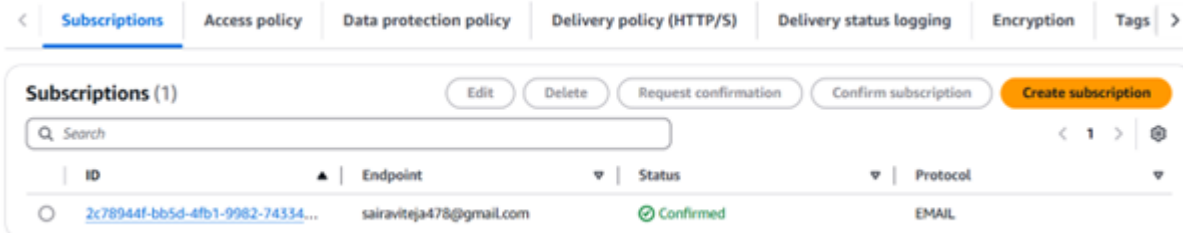
After your subscription is created, you must confirm it. [Info](#)

► **Subscription filter policy - optional** [Info](#)
This policy filters the messages that a subscriber receives.

► **Redrive policy (dead-letter queue) - optional** [Info](#)
Send undeliverable messages to a dead-letter queue.

[Cancel] [Create subscription]

- After subscription request for the mail confirmation



< **Subscriptions** | Access policy | Data protection policy | Delivery policy (HTTP/S) | Delivery status logging | Encryption | Tags >

Subscriptions (1) [Edit] [Delete] [Request confirmation] [Confirm subscription] [Create subscription]

Search

ID	Endpoint	Status	Protocol
2c78944f-bb5d-4fb1-9982-74334...	sairaviteja478@gmail.com	Confirmed	EMAIL

- Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail.

AWS Notification - Subscription Confirmation Inbox x

AWS Notifications <no-reply@sns.amazonaws.com>
to me ▾

9

You have chosen to subscribe to the topic:

arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):

[Confirm subscription](#)

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-opt-out](#)

AWS Notifications <no-reply@sns.amazonaws.com>
to me ▾

You have chosen to subscribe to the topic:

arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications

To confirm this subscription, click or visit the link below (If this was in error no action is necessary):

[Confirm subscription](#)

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to [sns-opt-out](#)



Simple Notification Service

Subscription confirmed!

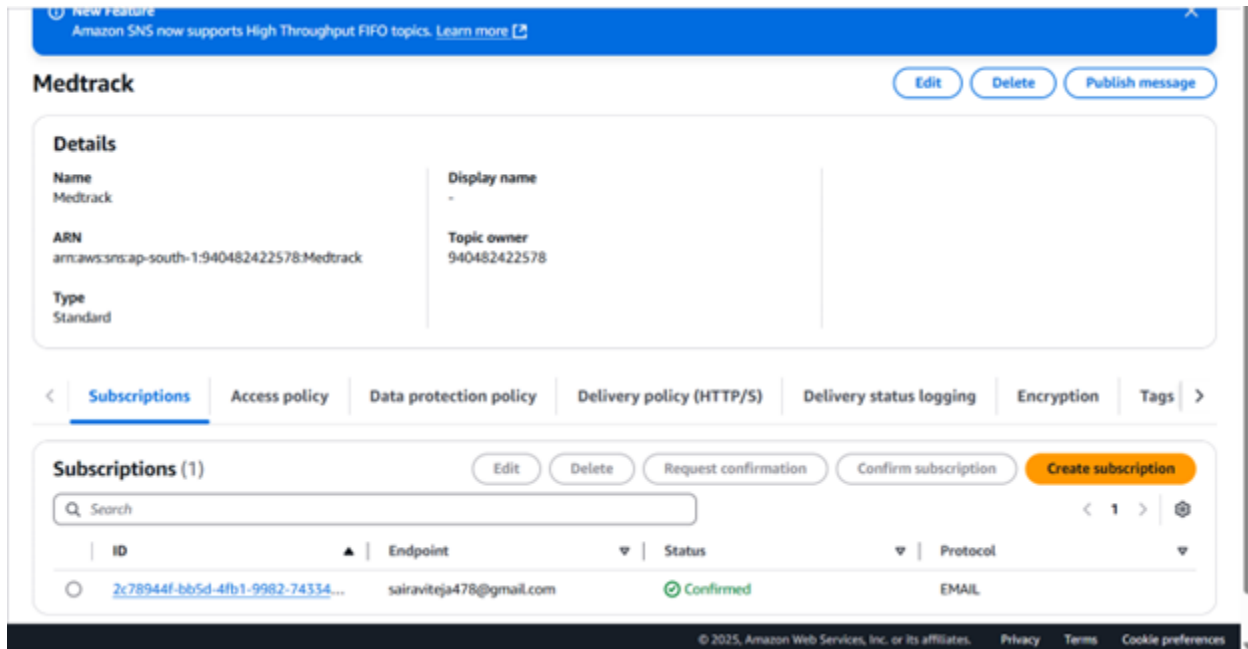
You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:ap-south-1:557690616836:BookRequestNotifications:d78e0371-9235-404d-952c-85c2743607c4

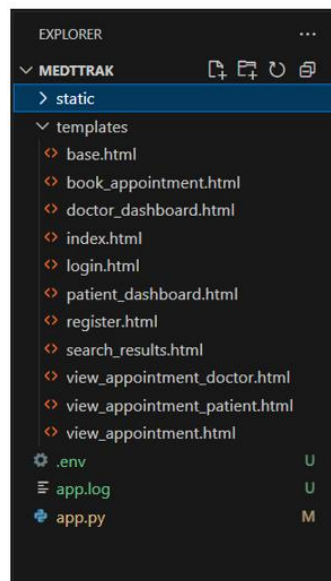
If it was not your intention to subscribe, [click here to unsubscribe](#).

- Successfully done with the SNS mail subscription and setup, now store the ARN link.



Milestone 4: Backend Development and Application Setup

- **Activity 4.1: Develop the backend using Flask**
 - File Explorer Structure:



Description: In the MedTrack project, the Flask app is initialized to establish the backend infrastructure, enabling it to handle multiple user interactions such as patient registration, appointment booking, and submission of medical reports. The Flask framework processes incoming requests, communicates with the DynamoDB database for storing user data, and integrates seamlessly with AWS services. Additionally, the routes and APIs are defined to manage different functionalities like secure login, appointment scheduling, and medical history retrieval. This initialization sets up the foundation for smooth, real-time communication between patients and doctors while ensuring the app is scalable and secure.

Description of the code :

- **Flask App Initialization**

In the MedTrack project, the Flask app is initialized to establish the backend infrastructure, enabling it to handle multiple user interactions such as patient registration, appointment booking, and submission of medical reports. The Flask framework processes incoming requests, communicates with the DynamoDB database for storing user data, and integrates seamlessly with AWS services. Additionally, the routes and APIs are defined to manage different functionalities like secure login, appointment scheduling, and medical history retrieval. This initialization sets up the foundation for smooth, real-time communication between patients and doctors while ensuring the app is scalable and secure.

```
app.py M X env U app.log U base.html view_appointment_doctor.html view_appointment_patient.html search_results.html
app.py > _
1 from flask import Flask, request, session, redirect, url_for, render_template, flash
2 import boto3
3 from werkzeug.security import generate_password_hash, check_password_hash
4 from datetime import datetime
5 import smtplib
6 from email.mime.text import MIMEText
7 from email.mime.multipart import MIMEMultipart
8 import logging
9 import os
10 import uuid
11 from dotenv import load_dotenv
12
13 # Load environment variables
14 load_dotenv()
15
16 # -----
17 # Flask App Initialization
18 # -----
19 app = Flask(__name__)
20 app.secret_key = os.environ.get('SECRET_KEY', 'temporary_key_for_development')
21
22 # -----
23 # App Configuration
24 # -----
25 AWS_REGION_NAME = os.environ.get('AWS_REGION_NAME', 'ap-south-1')
26
27 # Email Configuration
28 SMTP_SERVER = os.environ.get('SMTP_SERVER', 'smtp.gmail.com')
29 SMTP_PORT = int(os.environ.get('SMTP_PORT', 587))
30 SENDER_EMAIL = os.environ.get('SENDER_EMAIL')
31 SENDER_PASSWORD = os.environ.get('SENDER_PASSWORD')
32 ENABLE_EMAIL = os.environ.get('ENABLE_EMAIL', 'False').lower() == 'true'
33
34 # Table Names from .env
35 USERS_TABLE_NAME = os.environ.get('USERS_TABLE_NAME', 'UsersTable')
36 APPOINTMENTS_TABLE_NAME = os.environ.get('APPOINTMENTS_TABLE_NAME', 'AppointmentsTable')
37
```



```
app = Flask(__name__)
```

Description: Use **boto3** to connect to **DynamoDB** for handling user registration, book requests database operations and also mention `region_name` where Dynamodb tables are created.

SNS and Dynamodb initialization

In the MedTrack project, AWS SNS sends real-time notifications to patients and doctors about appointments and updates. DynamoDB stores user data, medical records, and appointments securely, offering fast, scalable access. Both services are integrated with Flask to ensure smooth communication and efficient data management.

```
app.py M X .env U app.log U base.html view_appointm
app.py > ...
38 # SNS Configuration
39 SNS_TOPIC_ARN = os.environ.get('SNS_TOPIC_ARN')
40 ENABLE_SNS = os.environ.get('ENABLE_SNS', 'False').lower() == 'true'
41
42 # -----
43 # AWS Resources
44 # -----
45 dynamodb = boto3.resource('dynamodb', region_name=AWS_REGION_NAME)
46 sns = boto3.client('sns', region_name=AWS_REGION_NAME)
47
48 # DynamoDB Tables
49 user_table = dynamodb.Table(USERS_TABLE_NAME)
50 appointment_table = dynamodb.Table(APPOINTMENTS_TABLE_NAME)
51
52 # -----
53 # Logging
54 # -----
55 logging.basicConfig(
56     level=logging.INFO,
57     format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
58     handlers=[
59         logging.FileHandler("app.log"),
60         logging.StreamHandler()
61     ]
62 )
63 logger = logging.getLogger(__name__)
64
65 # -----
66 # Helper Functions
67 # -----
68 def is_logged_in():
69     return 'email' in session
70
71 def get_user_role(email):
72     try:
73         response = user_table.get_item(Key={'email': email})
```

SNS Connection

Configure **SNS** to send notifications when a book request is submitted. Paste your stored ARN link in the `sns_topic_arn` space, along with the `region_name` where the SNS topic is created. Also, specify the chosen email service in `SMTP_SERVER` (e.g., Gmail, Yahoo, etc.) and enter the subscribed email in the `SENDER_EMAIL` section. Create an 'App password' for the email ID and store it in the `SENDER_PASSWORD` section.

```

app.py M X .env U app.log U base.html view_appointment_doctor.html
app.py > ...
71 def get_user_role(email):
72     ...
73     ...
74     ...
75     except Exception as e:
76         logger.error(f"Error fetching role: {e}")
77     return None
78
79 def send_email(to_email, subject, body):
80     if not ENABLE_EMAIL:
81         logger.info(f"[Email Skipped] Subject: {subject} to {to_email}")
82         return
83
84     try:
85         msg = MIMEText(body, 'plain')
86         msg['From'] = SENDER_EMAIL
87         msg['To'] = to_email
88         msg['Subject'] = subject
89         msg.attach(MIMEText(body, 'plain'))
90
91         server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
92         server.starttls()
93         server.login(SENDER_EMAIL, SENDER_PASSWORD)
94         server.sendmail(SENDER_EMAIL, to_email, msg.as_string())
95         server.quit()
96
97         logger.info(f"Email sent to {to_email}")
98     except Exception as e:
99         logger.error(f"Email sending failed: {e}")
100
101 def publish_to_sns(message, subject="Salon Notification"):
102     if not ENABLE_SNS:
103         logger.info("[SNS Skipped] Message: {}".format(message))
104         return
105
106     try:
107         response = sns.publish(
108             TopicArn=SNS_TOPIC_ARN,
109             Message=message,

```

```

        Message=message,
        Subject=subject
    )
    logger.info(f"SNS published: {response['MessageId']}")
except Exception as e:
    logger.error(f"SNS publish failed: {e}")

```

Routes for Web Pages

Register User:

The **login route** handles user input for authentication, verifying credentials against stored data in **DynamoDB**. On successful login, it increments the **login count** and redirects the user to the appropriate dashboard. This process ensures secure and efficient access to the platform.

```

124 # Home Page
125 @app.route('/')
126 def index():
127     if is_logged_in():
128         return redirect(url_for('dashboard'))
129     return render_template('index.html')
130
131 # Register User (Doctor/Patient)
132 @app.route('/register', methods=['GET', 'POST'])
133 def register():
134     if is_logged_in(): # Check if already logged in
135         return redirect(url_for('dashboard'))
136     if request.method == 'POST':
137         # Form validation
138         required_fields = ['name', 'email', 'password', 'age', 'gender', 'role']
139         for field in required_fields:
140             if field not in request.form or not request.form[field]:
141                 flash('Please fill in the (field) field', 'danger')
142                 return render_template('register.html')
143
144         # Check if passwords match
145         if 'confirm_password' in request.form and request.form['password'] != request.form['confirm_password']:
146             flash('Passwords do not match', 'danger')
147             return render_template('register.html')
148
149         name = request.form['name']
150         email = request.form['email']
151         password = generate_password_hash(request.form['password']) # Hash password
152         age = request.form['age']
153         gender = request.form['gender']
154         role = request.form['role'] # 'doctor' or 'patient'
155
156         # Check if user already exists
157         existing_user = user_table.get_item(Key={'email': email}).get('item')
158         if existing_user:
159             flash('Email already registered', 'danger')
160             return render_template('register.html')

```

• Login Route (GET/POST):

The login route handles user authentication by verifying credentials stored in DynamoDB. Upon successful login, it increments the login count and redirects the user to their dashboard. This ensures secure access to the platform while maintaining user activity logs.

```

180 # Send welcome email if enabled
181 def register():
182     if ENABLE_EMAIL:
183         welcome_msg = f'Welcome to Healthcare App, {name}! Your account has been created successfully.'
184         send_email(email, 'Welcome to Healthcare App', welcome_msg)
185
186     # Send admin notification via SMS if configured
187     if app.config.get('SMS_TOPIC_ARN'):
188         try:
189             sns.publish(
190                 TopicArn=app.config.get('SMS_TOPIC_ARN'),
191                 Message=f'New user registered: {name} ({email}) as {role}',
192                 Subject='New User Registration - Healthcare App'
193             )
194         except Exception as e:
195             logger.error(f'Failed to publish to SMS: {e}')
196
197     flash('Registration successful. Please log in.', 'success')
198     return redirect(url_for('login'))
199     return render_template('register.html')
200
201 # Login User (Doctor/Patient)
202 @app.route('/login', methods=['GET', 'POST'])
203 def login():
204     if is_logged_in(): # If the user is already logged in, redirect to dashboard
205         return redirect(url_for('dashboard'))
206
207     if request.method == 'POST':
208         if not request.form.get('email') or not request.form.get('password') or not request.form.get('role'):
209             flash('All fields are required', 'danger')
210             return render_template('login.html')
211
212         email = request.form['email']
213         password = request.form['password']
214         role = request.form['role'] # Get the selected role (doctor or patient)
215
216         # Validate user credentials:

```

```

app.py M X env U app.log U base.html view_appointment_doctor.html view_appointment_patient.html
app.py > register
def login():
    216 user = user_table.get_item(Key={'email': email}).get('Item')
    217
    218 if user:
    219     # Check password and role
    220     if check_password_hash(user['password'], password): # Use check_password_hash to verify hashed password
    221         if user['role'] == role:
    222             session['email'] = email
    223             session['role'] = role # Store the role in the session
    224             session['name'] = user.get('name', '')
    225
    226             # Update login count
    227             try:
    228                 user_table.update_item(
    229                     Key={'email': email},
    230                     UpdateExpression='set login_count = if_not_exists(login_count, :zero) + :inc',
    231                     ExpressionAttributeValues={':inc': 1, ':zero': 0}
    232                 )
    233             except Exception as e:
    234                 logger.error(f"Failed to update login count: {e}")
    235
    236             flash('login successful.', 'success')
    237             return redirect(url_for('dashboard'))
    238         else:
    239             flash('Invalid role selected.', 'danger')
    240     else:
    241         flash('Invalid password.', 'danger')
    242     else:
    243         flash('Email not found.', 'danger')
    244
    245     return render_template('login.html')
    246
    247 # Logout User
    248 @app.route('/logout')
    249 def logout():
    250     session.pop('email', None)
    251     session.pop('role', None)

```

Logout Route:

The logout functionality allows users to securely end their session, clearing any session data and redirecting them to the login page. The dashboard provides users with an overview of their activities, such as upcoming appointments for patients or patient records for doctors, with relevant actions based on user roles.

```

app.py M X env U app.log U base.html view_appointment_doctor.html
app.py > register
def logout():
    253 flash('You have been logged out.', 'success')
    254 return redirect(url_for('index'))
    255
    256 # Dashboard for both Doctors and Patients
    257 @app.route('/dashboard')
    258 def dashboard():
    259     if not is_logged_in():
    260         flash('Please log in to continue.', 'danger')
    261         return redirect(url_for('login'))
    262
    263     role = session['role']
    264     email = session['email']
    265
    266     if role == 'doctor':
    267         # Use GSI instead of scan for better performance
    268         try:
    269             response = appointment_table.query(
    270                 IndexName='doctormailIndex',
    271                 KeyConditionExpression='doctor_email = :email',
    272                 ExpressionAttributeValues={':email': email}
    273             )
    274             appointments = response['Items']
    275         except Exception as e:
    276             logger.error(f"Failed to fetch appointments: {e}")
    277             # Fallback to scan if GSI is not yet created
    278             try:
    279                 response = appointment_table.scan(
    280                     FilterExpression='doctor_email = :email',
    281                     ExpressionAttributeNames={'doctor_email': 'doctor_email'},
    282                     ExpressionAttributeValues={':email': email}
    283                 )
    284                 appointments = response['Items']
    285             except Exception as e:
    286                 logger.error(f"Fallback scan failed: {e}")
    287             appointments = []
    288

```

```

app.py M X env U app.log U base.html view_appointment_doctor.html
app.py > register
288 def dashboard():
289     return render_template('doctor_dashboard.html', appointments=appointments)
290
291 elif role == 'patient':
292     # Use GSI instead of scan for better performance
293     try:
294         response = appointment_table.query(
295             IndexName='PatientEmailIndex',
296             KeyConditionExpression="patient_email = :email",
297             ExpressionAttributeValues={":email": email})
298     )
299     appointments = response['Items']
300 except Exception as e:
301     logger.error(f"Failed to query appointments: {e}")
302     # Fallback to scan if GSI is not yet created
303     try:
304         response = appointment_table.scan(
305             FilterExpression="#patient_email = :email",
306             ExpressionAttributeNames={"#patient_email": "patient_email"},
307             ExpressionAttributeValues={":email": email})
308     )
309     appointments = response['Items']
310 except Exception as ex:
311     logger.error(f"Fallback scan failed: {ex}")
312     appointments = []
313
314 # Get list of doctors for booking new appointments
315 try:
316     doctor_response = user_table.scan(
317         FilterExpression="#role = :role",
318         ExpressionAttributeNames={"#role": "role"},
319         ExpressionAttributeValues={":role": "doctor"})
320 )
321 doctors = doctor_response['Items']
322 except Exception as e:
323     logger.error(f"Failed to fetch doctors: {e}")
324     doctors = []

```

Book Appointment Route:

The book appointment route allows users to select a date, time, and doctor for their appointment. Upon submission, the system stores the appointment details in DynamoDB and sends a confirmation notification via SNS. This ensures smooth scheduling and timely updates for both patients and doctors.

```

app.py M X env U app.log U base.html view_appointment_doctor.html view_ap
app.py > register
328 # Book an Appointment (Patient)
329 @app.route('/book_appointment', methods=['GET', 'POST'])
330 def book_appointment():
331     if not is_logged_in() or session['role'] != 'patient':
332         flash('Only patients can book appointments.', 'danger')
333         return redirect(url_for('login'))
334
335     if request.method == 'POST':
336         # Form validation
337         if not request.form.get('doctor_email') or not request.form.get('symptoms'):
338             flash('Please fill all required fields', 'danger')
339             return redirect(url_for('book_appointment'))
340
341         doctor_email = request.form['doctor_email']
342         symptoms = request.form['symptoms']
343         appointment_date = request.form.get('appointment_date', datetime.now().isoformat())
344         patient_email = session['email']
345
346         # Get patient and doctor information for notifications
347         try:
348             patient = user_table.get_item(Key={'email': patient_email}).get('item', {})
349             doctor = user_table.get_item(Key={'email': doctor_email}).get('item', {})
350
351             patient_name = patient.get('name', 'Patient')
352             doctor_name = doctor.get('name', 'Doctor')
353
354             # Create a new appointment
355             appointment_id = str(uuid.uuid4())
356             appointment_item = {
357                 'appointment_id': appointment_id,
358                 'doctor_email': doctor_email,
359                 'doctor_name': doctor_name,
360                 'patient_email': patient_email,
361                 'patient_name': patient_name,
362                 'symptoms': symptoms,
363                 'status': 'pending',

```

```

330 def book_appointment():
331     appointment_data = appointment_data
332     'created_at': datetime.now().isoformat(),
333 }
334
335 appointment_table.put_item(item=appointment_item)
336
337 # Send email notification if enabled
338 if EMAIL_ENABLED:
339     # Send email notification to doctor
340     doctor_email = "Dear Dr. {doctor_name},\nA new appointment has been booked by {patient_name}.\nSymptoms: {symptoms}\nDate: {appointment_data}\nPlease login
341     send_email(doctor_email, "New Appointment Notification", doctor_email)
342
343     # Send confirmation email to patient
344     patient_email = "Dear {patient_name},\nYour appointment with Dr. {doctor_name} has been booked successfully.\nDate: {appointment_data}\nThank you for using
345     send_email(patient_email, "Appointment Confirmation", patient_email)
346
347 # Send SMS notification if configured
348 if app.config.get('SMS_NOTIFICATION'):
349     try:
350         sns.publish(
351             TopicArn=app.config.get('SNS_TOPIC_ARN'),
352             Message="New appointment booked online {patient_name} with Dr. {doctor_name} for date {appointment_data}.",
353             Subject="New Appointment Booked - HealthCare App"
354         )
355     except Exception as e:
356         logger.error(f"Failed to publish to SNS: {e}")
357
358     flash("Appointment booked successfully.", 'success')
359     return redirect(url_for('user_dashboard'))
360 except Exception as e:
361     logger.error(f"Failed to book appointment: {e}")
362     flash("An error occurred while booking the appointment. Please try again.", 'danger')
363     return redirect(url_for('book_appointment'))
364
365 # Get list of doctors for selection
366 try:
367     response = user_table.scan(
368         FilterExpression="#role = :role",
369         ExpressionAttributeNames={"#role": "role"},
370         ExpressionAttributeValues={"":role": "doctor"}
371     )
372     doctors = response['Items']
373 except Exception as e:
374     logger.error(f"Failed to fetch doctors: {e}")
375     doctors = []
376
377 return render_template('book_appointment.html', doctors=doctors)
378
379 @app.route('/view_appointment/<appointment_id>', methods=['GET', 'POST'])
380 def view_appointment(appointment_id):
381     if not is_logged_in():
382         flash("Please log in to continue.", 'danger')
383         return redirect(url_for('login'))
384
385     # Fetch appointment details
386     try:
387         response = appointment_table.get_item(Key={'appointment_id': appointment_id})
388         appointment = response.get('Item')
389
390         if not appointment:
391             flash("Appointment not found.", 'danger')
392             return redirect(url_for('dashboard'))
393
394         # Security check - verify the logged-in user should access this appointment
395         if session['role'] == 'doctor' and appointment['doctor_email'] != session['email']:
396             flash("You are not authorized to view this appointment.", 'danger')
397             return redirect(url_for('dashboard'))
398         elif session['role'] == 'patient' and appointment['patient_email'] != session['email']:
399             flash("You are not authorized to view this appointment.", 'danger')
400             return redirect(url_for('dashboard'))
401
402         if request.method == 'POST' and session['role'] == 'doctor':
403             diagnosis = request.form['diagnosis']
404             treatment_plan = request.form['treatment_plan']
405             prescription = request.form['prescription']
406
407             # Update appointment with doctor's diagnosis and treatment plan
408             appointment_table.update_item(
409                 Key={'appointment_id': appointment_id},

```

View Appointment Route:

The view appointment route allows users to access a list of their upcoming appointments. It retrieves appointment details from DynamoDB and displays them on the user's dashboard, providing information like date, time, and doctor. This ensures users can easily track their scheduled appointments.

```

330 def book_appointment():
331     try:
332         response = user_table.scan(
333             FilterExpression="#role = :role",
334             ExpressionAttributeNames={"#role": "role"},
335             ExpressionAttributeValues={"":role": "doctor"}
336         )
337         doctors = response['Items']
338     except Exception as e:
339         logger.error(f"Failed to fetch doctors: {e}")
340         doctors = []
341     return render_template('book_appointment.html', doctors=doctors)
342
343 @app.route('/view_appointment/<appointment_id>', methods=['GET', 'POST'])
344 def view_appointment(appointment_id):
345     if not is_logged_in():
346         flash("Please log in to continue.", 'danger')
347         return redirect(url_for('login'))
348
349     # Fetch appointment details
350     try:
351         response = appointment_table.get_item(Key={'appointment_id': appointment_id})
352         appointment = response.get('Item')
353
354         if not appointment:
355             flash("Appointment not found.", 'danger')
356             return redirect(url_for('dashboard'))
357
358         # Security check - verify the logged-in user should access this appointment
359         if session['role'] == 'doctor' and appointment['doctor_email'] != session['email']:
360             flash("You are not authorized to view this appointment.", 'danger')
361             return redirect(url_for('dashboard'))
362         elif session['role'] == 'patient' and appointment['patient_email'] != session['email']:
363             flash("You are not authorized to view this appointment.", 'danger')
364             return redirect(url_for('dashboard'))
365
366         if request.method == 'POST' and session['role'] == 'doctor':
367             diagnosis = request.form['diagnosis']
368             treatment_plan = request.form['treatment_plan']
369             prescription = request.form['prescription']
370
371             # Update appointment with doctor's diagnosis and treatment plan
372             appointment_table.update_item(
373                 Key={'appointment_id': appointment_id},

```


Search Appointment Route:

The search appointment route enables users to search for specific appointments by date, doctor, or status. It queries DynamoDB to fetch relevant appointment details and displays the results in real-time, allowing users to quickly find specific appointments from their history or upcoming schedule.

```

411 view_appointment(appointment_id):
412     ExpressionAttributeValues={
413         'id': diagnosis,
414         't': treatment_plan,
415         'p': prescription,
416         's': 'completed',
417         'u': datetime.now().isoformat()
418     },
419     ExpressionAttributeNames={
420         's': 'status' # Use s as an alias for the reserved keyword 'status'
421     }
422 )
423
424 # Send email notification to patient if enabled
425 if ENABLE_EMAIL:
426     patient_email = appointment['patient_email']
427     patient_name = appointment.get('patient_name', 'Patient')
428     doctor_name = appointment.get('doctor_name', 'your doctor')
429
430     patient_msg = f'Dear {patient_name},\nYour appointment with Dr. {doctor_name} has been completed.\nDiagnosis: {diagnosis}\nTreatment Plan: {treatment_plan}'
431     send_email(patient_email, "Appointment Completed - Diagnosis Available", patient_msg)
432
433     flash("Diagnosis submitted successfully.", "success")
434     return redirect(url_for('dashboard'))
435
436 # Determine which template to render based on user role
437 if session['role'] == 'doctor':
438     return render_template("view_appointment_doctor.html", appointment=appointment)
439 else:
440     return render_template("view_appointment_patient.html", appointment=appointment)
441
442 except Exception as e:
443     logger.error(f"Error in view_appointment: {e}")
444     flash("An error occurred. Please try again.", "danger")
445     return redirect(url_for('dashboard'))
446
447 # Search functionality for appointments
448 @route('/search_appointments', methods=['GET', 'POST'])
449 search_appointments():
450     if not is_logged_in():
451         flash("Please log in to continue.", "danger")
452         return redirect(url_for('login'))
453
454     if request.method == 'POST':
455         search_term = request.form.get('search_term', '')
  
```

Profile Route:

The profile route allows users to view and update their personal information, such as name, contact details, and medical history. It retrieves user data from DynamoDB and displays it on the dashboard. Users can also make changes to their profile, which are then saved securely in the database.

```

480 search_appointments():
481     try:
482         if session['role'] == 'doctor':
483             # Doctors can search their patients by name
484             response = appointment_table.scan(
485                 FilterExpression=~doctor_email = :email AND contains(:patient_name, :search_term),
486                 ExpressionAttributeNames={
487                     'doctor_email': 'doctor_email',
488                     'patient_name': 'patient_name'
489                 },
490                 ExpressionAttributeValues={
491                     ':email': session['email'],
492                     ':search': search_term
493                 }
494             )
495             # Patient
496             # Patients can search their appointments by doctor name or status
497             response = appointment_table.scan(
498                 FilterExpression=~patient_email = :email AND (contains(:doctor_name, :search_term) OR contains(:status, :search_term)),
499                 ExpressionAttributeNames={
500                     'patient_email': 'patient_email',
501                     'doctor_name': 'doctor_name',
502                     'status': 'status'
503                 },
504                 ExpressionAttributeValues={
505                     ':email': session['email'],
506                     ':search': search_term
507                 }
508             )
509
510             appointments = response['Items']
511             return render_template("search_results.html", appointments=appointments, search_term=search_term)
512         except Exception as e:
513             logger.error(f"Search failed: {e}")
514             flash("Search failed. Please try again.", "danger")
515             return redirect(url_for('dashboard'))
516
517 # Profile page
518 @route('/profile', methods=['GET', 'POST'])
519 profile():
520     if not is_logged_in():
521         flash("Please log in to continue.", "danger")
522         return redirect(url_for('login'))
  
```

Submit Diagnosis Route:

The submit diagnosis route allows users (patients) to upload and submit their diagnosis reports. Upon submission, the system stores the diagnosis data in DynamoDB and notifies the assigned doctor via SNS. This ensures that medical reports are securely recorded and shared for timely review and treatment.

```

app.py M X .env U app.log U base.html view_appointment_doctor.h
app.py > register
527 profile():
528     email = session['email']
529     try:
530         user = user_table.get_item(Key={'email': email}).get('Item', {})
531
532         if request.method == 'POST':
533             # Update user profile
534             name = request.form.get('name')
535             age = request.form.get('age')
536             gender = request.form.get('gender')
537
538             update_expression = "SET #name = :name, age = :age, gender = :gender"
539             expression_values = {
540                 ':name': name,
541                 ':age': age,
542                 ':gender': gender
543             }
544
545             # Update specialization only for doctors
546             if session['role'] == 'doctor' and 'specialization' in request.form:
547                 update_expression += ", specialization = :spec"
548                 expression_values[':spec'] = request.form['specialization']
549
550             user_table.update_item(
551                 Key={'email': email},
552                 UpdateExpression=update_expression,
553                 ExpressionAttributeValues=expression_values,
554                 ExpressionAttributeNames={'#name': 'name'})
555
556             # Update session name
557             session['name'] = name
558
559             flash('Profile updated successfully.', 'success')
560             return redirect(url_for('profile'))
561
562         return render_template('profile.html', user=user)
563     except Exception as e:
564         logger.error(f"Profile error: {e}")
565         flash('An error occurred. Please try again.', 'danger')
566         return redirect(url_for('dashboard'))
567
568 bmit Diagnosis (Doctor) - For form submission
569 .route('/submit_diagnosis/<appointment_id>', methods=['POST'])
570 submit_diagnosis(appointment_id):

```

Health Routing And Deployment Code :

The health routing feature in the MedTrack project checks the system's status by sending a request to a specific endpoint, ensuring the backend services are functioning properly. The name == '__main__' block is used in the Flask app to ensure that the application runs only if the script is executed directly, not when imported as a module, enabling local development or deployment on a server. This setup ensures that the app runs smoothly and is self-contained during execution.

```

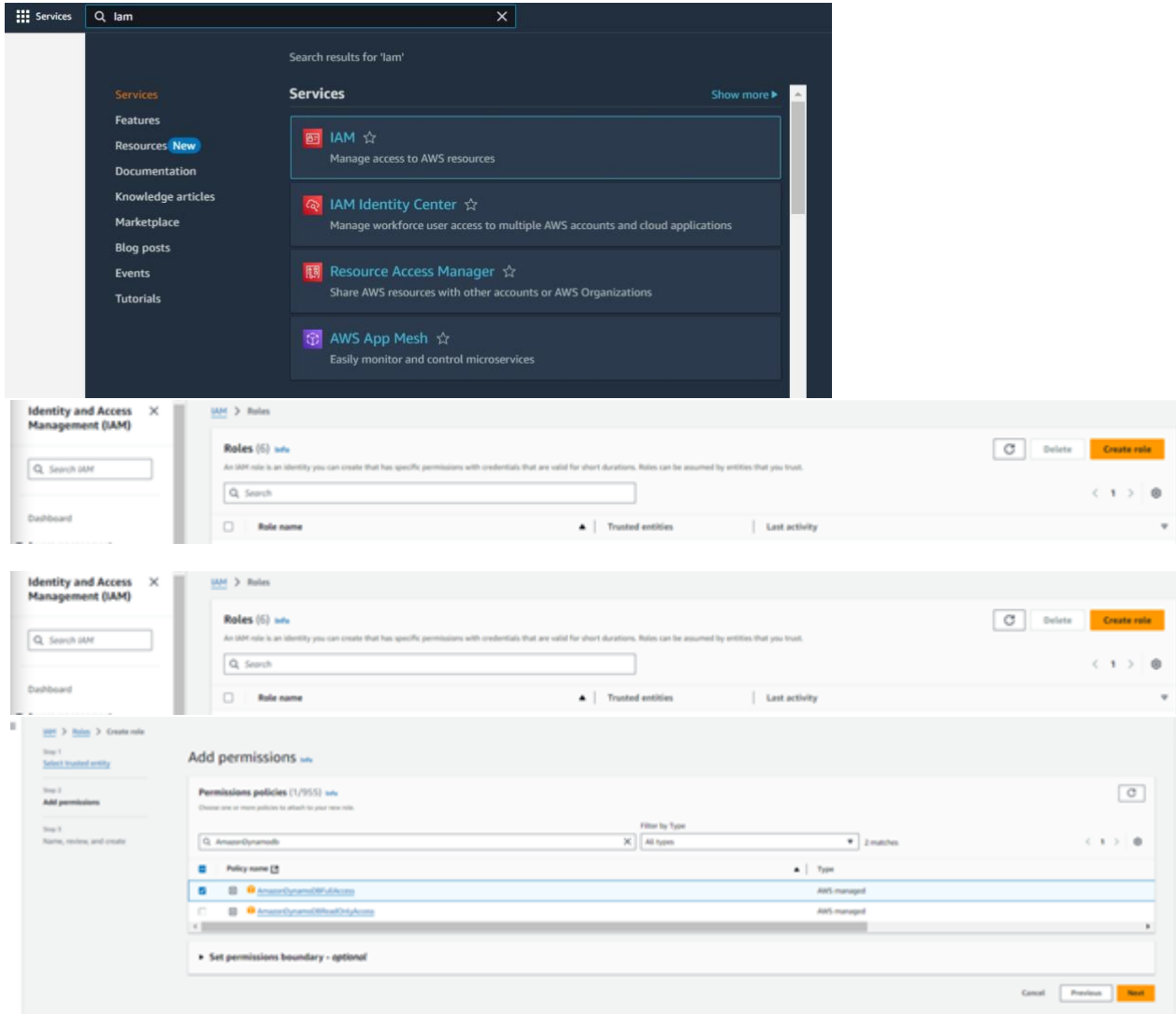
app.py | X | @ .env | Z app.py | home.html | view_appointment_doctor.html | view_appointment_patient.html | search_results.html | index.html | register.html
app.py > register
375 submit_diagnosis(appointment_id):
385     )
386
387     # Send email notification to patient if enabled
388     if ENABLE_EMAIL:
389         patient_email = appointment['patient_email']
390         patient_name = appointment.get('patient_name', 'Patient')
391         doctor_name = session.get('name', 'your doctor')
392
393         patient_msg = f'Dear {patient_name},\n\nYour appointment with Dr. {doctor_name} has been completed.\n\nDiagnosis: {diagnosis}\n\nTreatment Plan: {treatment_plan}\n\n'
394         send_email(patient_email, 'Appointment Completed - Diagnosis Available', patient_msg)
395
396         flash('Diagnosis submitted successfully.', 'success')
397         return redirect(url_for('dashboard'))
398 except Exception as e:
399     logger.error(f'Submit diagnosis error: {e}')
400     flash('An error occurred while submitting the diagnosis. Please try again.', 'danger')
401     return redirect(url_for('view_appointment', appointment_id=appointment_id))
402
403 # Health endpoint for AWS load balancers
404 @route('/health')
405 def health():
406     return {'status': 'healthy'}, 200
407
408 # The Flask app
409 if __name__ == '__main__':
410     port = int(os.environ.get('PORT', 5000))
411     debug_mode = os.environ.get('FLASK_ENV') == 'development'
412     app.run(host='0.0.0.0', port=port, debug=debug_mode)

```


Milestone 5: IAM Role Setup

- **Activity 5.1: Create IAM Role.**

- In the AWS Console, go to IAM and create a new IAM Role for EC2 to interact with DynamoDB and SNS.

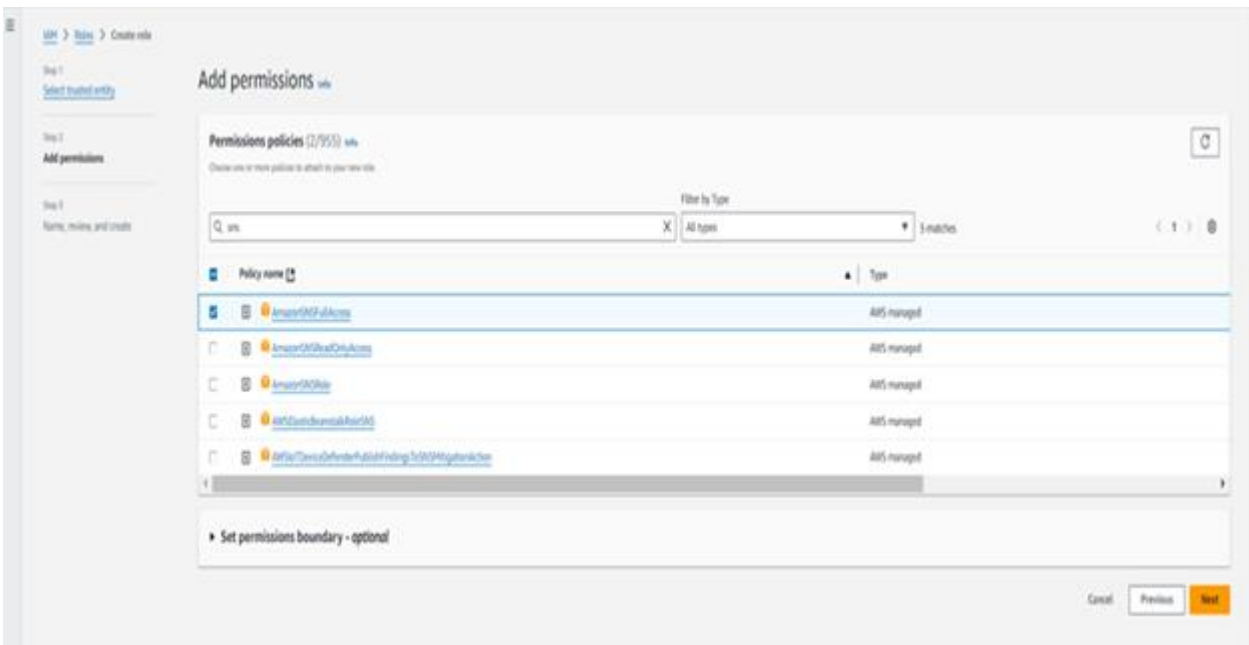


The first screenshot shows the AWS IAM console search results for 'iam'. The second screenshot shows the 'Create role' button. The third screenshot shows the 'Add permissions' step, where the 'AmazonDynamoDBFullAccess' policy is selected.

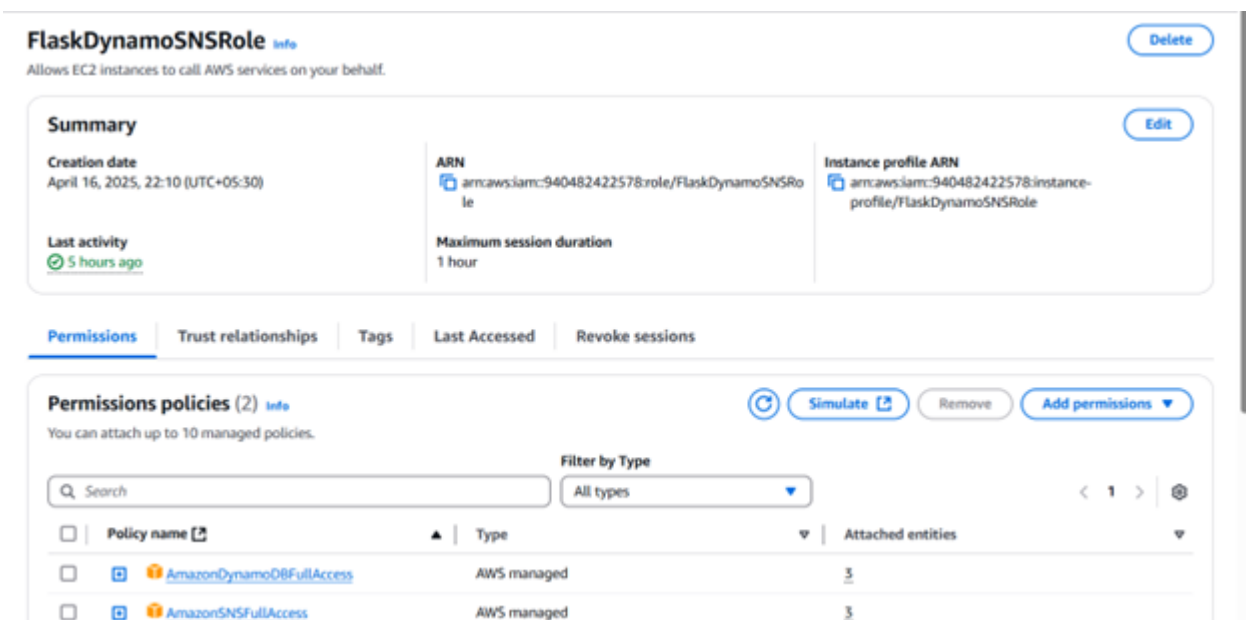
- **Activity 5.2: Attach Policies.**

Attach the following policies to the role:

- AmazonDynamoDBFullAccess: Allows EC2 to perform read/write operations on DynamoDB.
- AmazonSNSFullAccess: Grants EC2 the ability to send notifications via SNS.



To create a role named **flaskdynamodbsns**, go to the AWS IAM console, create a new role, and assign DynamoDBFullAccess and SNSFullAccess policies. Name the role flaskdynamodbsns and attach it to the necessary AWS services. This role will allow your Flask app to interact with both DynamoDB and SNS seamlessly.



Milestone 6: EC2 Instance Setup

- Note: Load your Flask app and Html files into GitHub repository.
- To upload your Flask app and HTML files to GitHub, initialize the repository using git init, commit the files with git add . && git commit -m "Initial commit", and push them using git push -u origin main. This will load your project onto GitHub.

 templates	pushes	2 days ago
 .env	Add files via upload	5 hours ago
 app.py	Update app.py	5 hours ago

Local

Codespaces

 Clone 

HTTPS

SSH

GitHub CLI

https://github.com/Ravi-teja-777/medtrack_app.git 

Clone using the web URL.

 Open with GitHub Desktop

 Download ZIP

[AWS-PROJECT](#) / [Medtrack](#) / [medtrack-project-main](#) / 

 **tejashwinivr** Add files via upload

Name	Last commit message
 ..	
 static/css	Add files via upload
 templates	Add files via upload
 app.py	Add files via upload

Local

Codespaces

 Clone 

HTTPS

SSH

GitHub CLI

[https://github.com/Ravi-teja-777/medtrack_app. !\[\]\(1fa906618d33c0f586cba67e92c6c1a9_img.jpg\)](https://github.com/Ravi-teja-777/medtrack_app.)

Clone using the web URL.

 Open with GitHub Desktop

 Download ZIP

 **main**  **1 Branch**  **0 Tags**

 Go to file

 **Code** 

 **tejashwinivr** Delete Medtrack-20260306T143320Z-3-001.zip

132286b · 3 days ago

 **3 Commits**

 Medtrack/medtrack-project-main

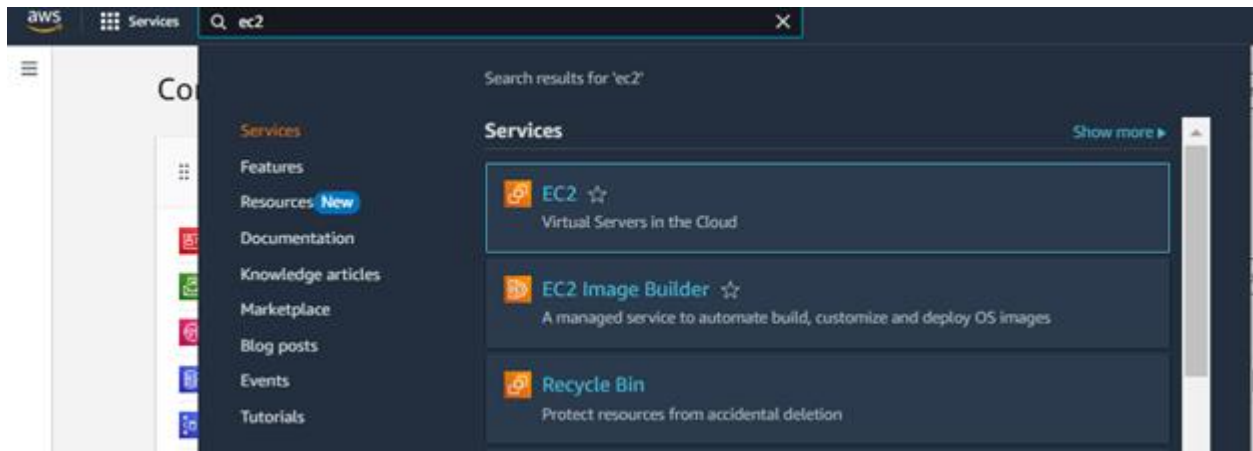
Add files via upload

3 days ago

Activity 6.1: Launch an EC2 instance to host the Flask application.

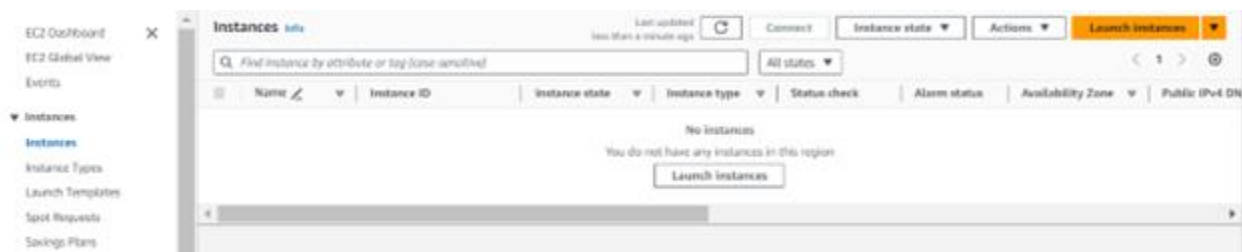
- **Launch EC2 Instance**

- In the AWS Console, navigate to EC2 and launch a new instance.



To launch an EC2 instance with the name **flaskec2role**, follow these steps:

- Go to the **AWS EC2 Dashboard** and click on **Launch Instance**.
- Select your desired AMI, instance type, configure instance details, and under **IAM role**, choose the role **flaskec2role**. Finally, launch the instance.



- Choose Amazon Linux 2 or Ubuntu as the AMI and t2.micro as the instance type (free-tier eligible).

EC2 > Instances > Launch an instance

Launch an instance info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags info

Name

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) info

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Search our full catalog including 1000s of application and OS images

Recents **Quick Start**

Amazon Linux
aws

macOS
Mac

Ubuntu
ubuntu

Windows
Microsoft

Red Hat
Red Hat

SUSE Linux
SUSE

Debian
debian

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

To launch an EC2 instance with **Amazon Linux 2** or **Ubuntu** as the AMI and **t2.micro** as the instance type (free-tier eligible):

- In the **Launch Instance** wizard, choose **Amazon Linux 2** or **Ubuntu** from the available AMIs.
- Select **t2.micro** as the instance type, which is free-tier eligible, and continue with the configuration and launch steps.

Amazon Linux
aws

macOS
Mac

Ubuntu
ubuntu

Windows
Microsoft

Red Hat
Red Hat

[Browse more AMIs](#)
Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 AMI Free tier eligible

ami-02b49a24cfb95941c (64-bit (x86), uefi-preferred) / ami-04ad8c7fcc828fad4 (64-bit (Arm), uefi)

Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Architecture

64-bit (x86)

Boot mode

uefi-preferred

AMI ID

ami-02b49a24cfb95941c

Verified provider

To create and download the key pair for server access:

- In the **Launch Instance** wizard, under the **Key Pair** section, click **Create a new key pair**.

- Name your key pair (e.g., **flaskkeypair**) and click **Download Key Pair**. This will download the .pem file to your system, which you will use to access the EC2 instance securely via SSH.

▼ Instance type Info | Get advice

Instance type

t2.microFree tier eligible

Family: t2 1 vCPU 1 GiB Memory Current generation: true
On-Demand Linux base pricing: 0.0124 USD per Hour
On-Demand Windows base pricing: 0.017 USD per Hour
On-Demand RHEL base pricing: 0.0268 USD per Hour
On-Demand SUSE base pricing: 0.0124 USD per Hour

☐ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

Select▼

 [Create new key pair](#)

▼ Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

linuxkeypair▼

 [Create new key pair](#)

Inbound Security Group Rules

▼ Security group rule 1 (TCP; 22, 0.0.0.0/0)

Type [Info](#)
ssh

Protocol [Info](#)
TCP

Port range [Info](#)
22

Source type [Info](#)
Anywhere

Source [Info](#)

Description - optional [Info](#)
e.g. SSH for admin desktop

[Remove](#)

▼ Security group rule 2 (TCP; 80, 0.0.0.0/0)

Type [Info](#)
HTTP

Protocol [Info](#)
TCP

Port range [Info](#)
80

Source type [Info](#)
Custom

Source [Info](#)

Description - optional [Info](#)
e.g. SSH for admin desktop

[Remove](#)

▼ Security group rule 3 (TCP; 5000, 0.0.0.0/0)

Type [Info](#)
Custom TCP

Protocol [Info](#)
TCP

Port range [Info](#)
5000

Source type [Info](#)
Custom

Source [Info](#)

Description - optional [Info](#)
e.g. SSH for admin desktop

[Remove](#)

[Add security group rule](#)

Launch an instance

Success
Successfully initiated launch of instance i-0218621010x290

Launch log

Next Steps

What would you like to do next with this instance, for example "create alarm" or "create budget"?

Create billing and free tier usage alerts

To manage costs and avoid surprise bills, set up email notifications for billing and free tier usage thresholds.

[Create billing alerts](#)

Connect to your instance

Once your instance is running, log into it from your local computer.

[Connect to instance](#)

[Learn more](#)

Connect an RDS database

Configure the connection between an EC2 instance and a database to allow traffic flow between them.

[Connect an RDS database](#)

[Create a new RDS database](#)

[Learn more](#)

Create EBS snapshot policy

Create a policy that automates the creation, retention, and deletion of EBS snapshots.

[Create EBS snapshot policy](#)

Manage detailed monitoring

Enable or disable detailed monitoring for the instance. If you enable detailed monitoring, the Amazon EC2 console displays monitoring graphs with a 1-minute period.

[Manage detailed monitoring](#)

Create Load Balancer

Create an application, network gateway or classic Elastic Load Balancer.

[Create Load Balancer](#)

Create AWS budget

AWS Budgets allow you to create budgets, forecast spend, and take action on your costs and usage from a single location.

[Create AWS budget](#)

Manage CloudWatch alarms

Create or update Amazon CloudWatch alarms for the instance.

[Manage CloudWatch alarms](#)

Disaster recovery for your instances

Recover the instances you just launched into a different Availability Zone or a different Region using AWS Elastic Disaster Recovery (EDR).

[Disaster recovery for your instances](#)

Monitor for suspicious runtime activities

AWS GuardDuty enables you to continuously monitor for malicious runtime activity and unauthorized behavior, with near real-time visibility into on-host activities occurring across your Amazon EC2 workloads.

[Monitor for suspicious runtime activities](#)

Get instance screenshot

Capture a screenshot from the instance and view it as an image. This is useful for troubleshooting or unresponsive instances.

[Get instance screenshot](#)

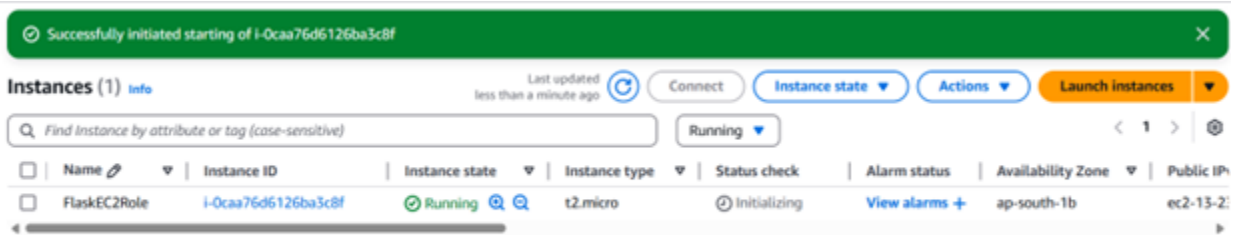
Get system log

View the instance's system log to troubleshoot items.

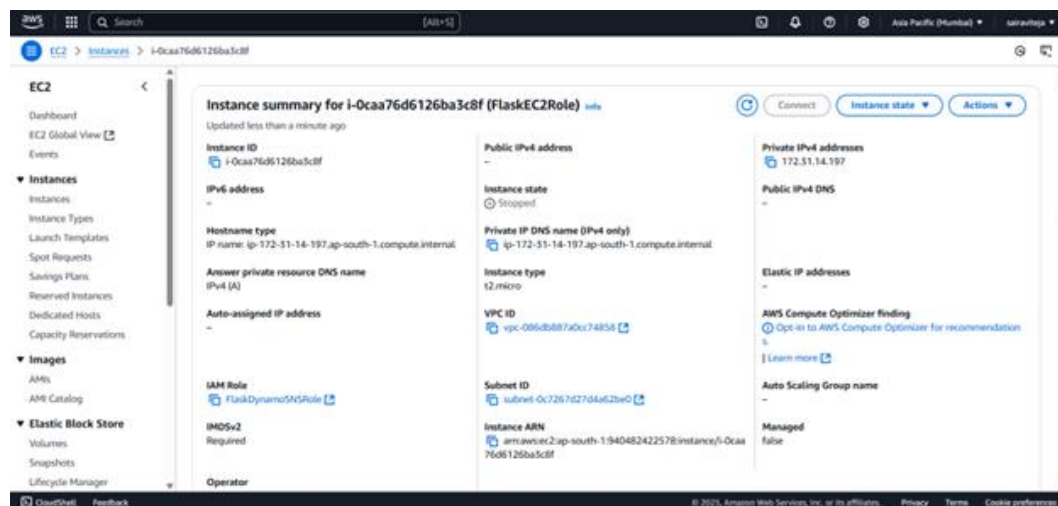
[Get system log](#)

[View all instances](#)

To connect to EC2 using EC2 Instance Connect, start by ensuring that an IAM role is attached to your EC2 instance. You can do this by selecting your instance, clicking on Actions, then navigating to Security and selecting Modify IAM Role to attach the appropriate role. After the IAM role is connected, navigate to the EC2 section in the AWS Management Console. Select the EC2 instance you wish to connect to. At the top of the EC2 Dashboard, click the Connect button. From the connection methods presented, choose EC2 Instance Connect. Finally, click Connect again, and a new browser-based terminal will open, allowing you to access your EC2 instance directly from your browser.



The EC2 instance you are launching is configured with Amazon Linux 2 or Ubuntu as the AMI, t2.micro as the instance type (free-tier eligible), and flaskec2role IAM role for appropriate permissions. The flaskkeypair key pair is created for secure server access via SSH, and the instance is set to auto-assign a public IP for internet accessibility. The security group is configured to allow SSH (port 22) access for remote login.



To modify the **IAM role** for your EC2 instance:

- Go to the **AWS IAM Console**, select **Roles**, and find the **flaskec2role**.
- Click **Attach Policies**, then choose the required policies (e.g., **DynamoDBFullAccess**, **SNSFullAccess**) and click **Attach Policy**.
- If needed, update the instance to use this modified role by selecting the EC2 instance, clicking **Actions**, then **Security**, and **Modify IAM role** to select the updated role.



Modify IAM role [info](#)

Attach an IAM role to your instance.

Instance ID
i-0caa76d6126ba3c8f (FlaskEC2Role)

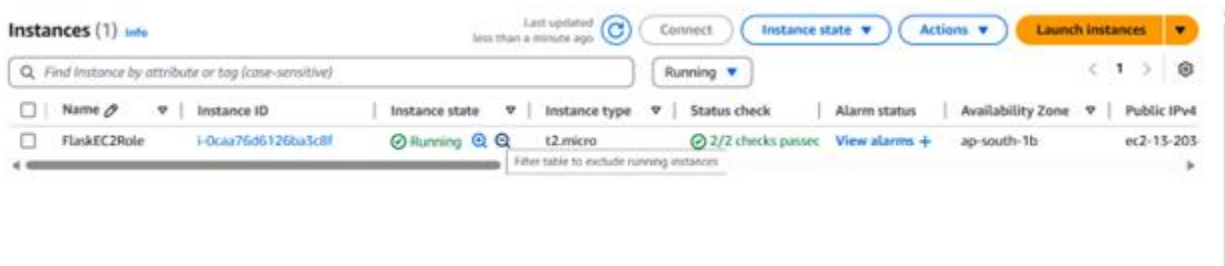
IAM role
Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.
FlaskDynamoSNSRole

[Create new IAM role](#)

[Cancel](#) [Update IAM role](#)

To connect to your EC2 instance:

- Go to the **EC2 Dashboard**, select your running instance, and click **Connect**.
- Follow the instructions provided in the **Connect To Your Instance** dialog, which will show the SSH command (e.g., `ssh -i flaskkeypair.pem ec2-user@<public-ip>`) to access your instance using the downloaded .pem key.



Instances (1) [info](#)

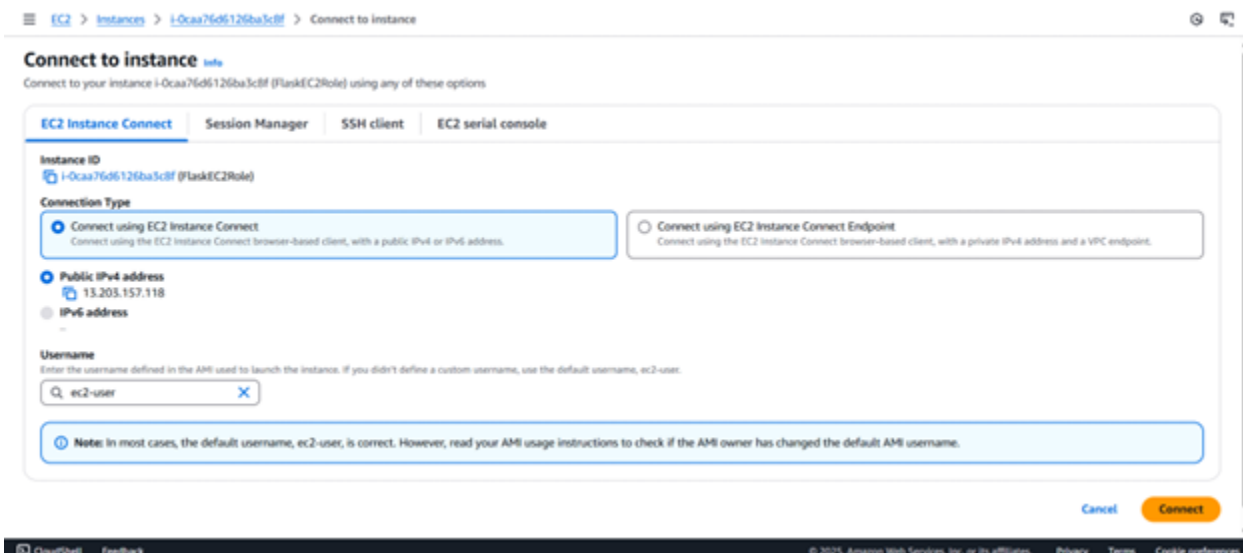
Find Instance by attribute or tag (case-sensitive)

Running

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☐	FlaskEC2Role	i-0caa76d6126ba3c8f	Running	t2.micro	2/2 checks passed	View alarms	ap-south-1b	ec2-13-203-157-118

[Filter table to exclude running instances](#)

- Now connect the EC2 with the files



Connect to instance [info](#)

Connect to your instance i-0caa76d6126ba3c8f (FlaskEC2Role) using any of these options

EC2 Instance Connect | Session Manager | SSH client | EC2 serial console

Instance ID
i-0caa76d6126ba3c8f (FlaskEC2Role)

Connection Type

☒ Connect using EC2 Instance Connect
Connect using the EC2 Instance Connect browser-based client, with a public IPv4 or IPv6 address.

☐ Connect using EC2 Instance Connect Endpoint
Connect using the EC2 Instance Connect browser-based client, with a private IPv4 address and a VPC endpoint.

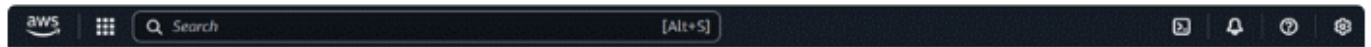
☒ Public IPv4 address
13.203.157.118

☐ IPv6 address

Username
Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user.
ec2-user

Note: In most cases, the default username, ec2-user, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

[Cancel](#) [Connect](#)



```
A newer release of "Amazon Linux" is available.
Version 2023.7.20250414:
Run "/usr/bin/dnf check-release-update" for full release and version update info

      _ _ _ _ _
     _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/
    _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/
   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/
  _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/
 _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/
_/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/   _/

Amazon Linux 2023

https://aws.amazon.com/linux/amazon-linux-2023

Last login: Fri Apr 18 14:42:17 2025 from 13.233.177.3
[ec2-user@ip-172-31-14-197 ~]$
```

i-0caa76d6126ba3c8f (FlaskEC2Role)

PublicIPs: 13.203.157.118 PrivateIPs: 172.31.14.197

Milestone 7: Deployment on EC2

Activity 7.1: Install Software on the EC2 Instance

Update and install Python + pip

```
sudo apt update sudo apt install
```

```
python3-pip -y sudo apt install git -
```

```
y sudo apt install python3-venv -y
```

```
python3 -m venv venv source
```

```
venv/bin/activate
```

Activity 7.2: Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

Run: 'git clone <https://github.com/tejashwinivr/AWS-PROJECT.git> Note: change your-github-username and your-repository-name with your credentials here: '<https://github.com/tejashwinivr/AWS-PROJECT.git>'

- This will download your project to the EC2 instance.

To navigate to the project directory, run the following command:

```
cd <your-folder>
```

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

```
sns = boto3.client('sns', region_name=REGION)
```

```
git pull origin main ( Only if you made changes in git and want to reflect them in the EC2 terminal.)
```

Install requirements

```
pip install flask boto3
```

Launch the Flask app:

```
sudo -E venv/bin/python3 app.py
```

Verify the Flask app is running: `http://your-ec2-public-ip`

- Run the Flask app on the EC2 instance

Access the website through:

PublicIPs: <https://github.com/tejashwinivr/AWS-PROJECT.git>

Milestone 8: Testing and Deployment

- **Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.**

Home Page: Conduct functional testing to verify user registration, login, book requests, and notifications.

Home Page:

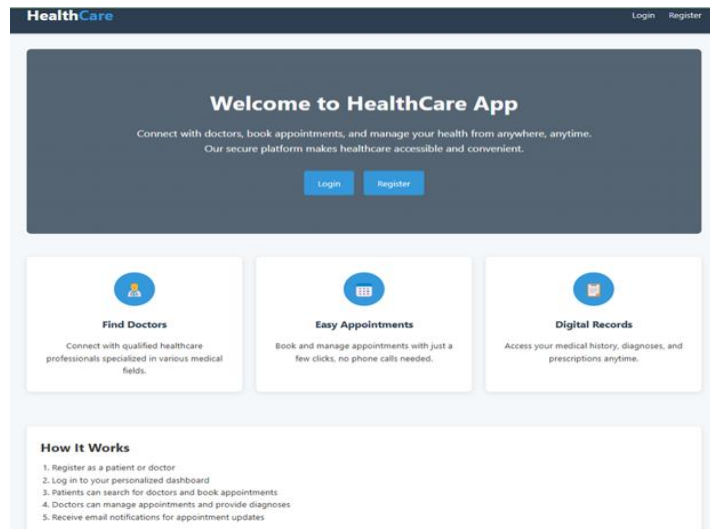
The Home Page of your project is the main entry point for users, where they can interact with the system. It typically includes:

Input Fields: For users to enter basic information like appointment requests, diagnosis submissions, or service bookings.

Navigation: Links to other sections such as the login page, dashboard, or service options.

Responsive Design: Ensures the page is accessible across devices with a clean, user-friendly interface.

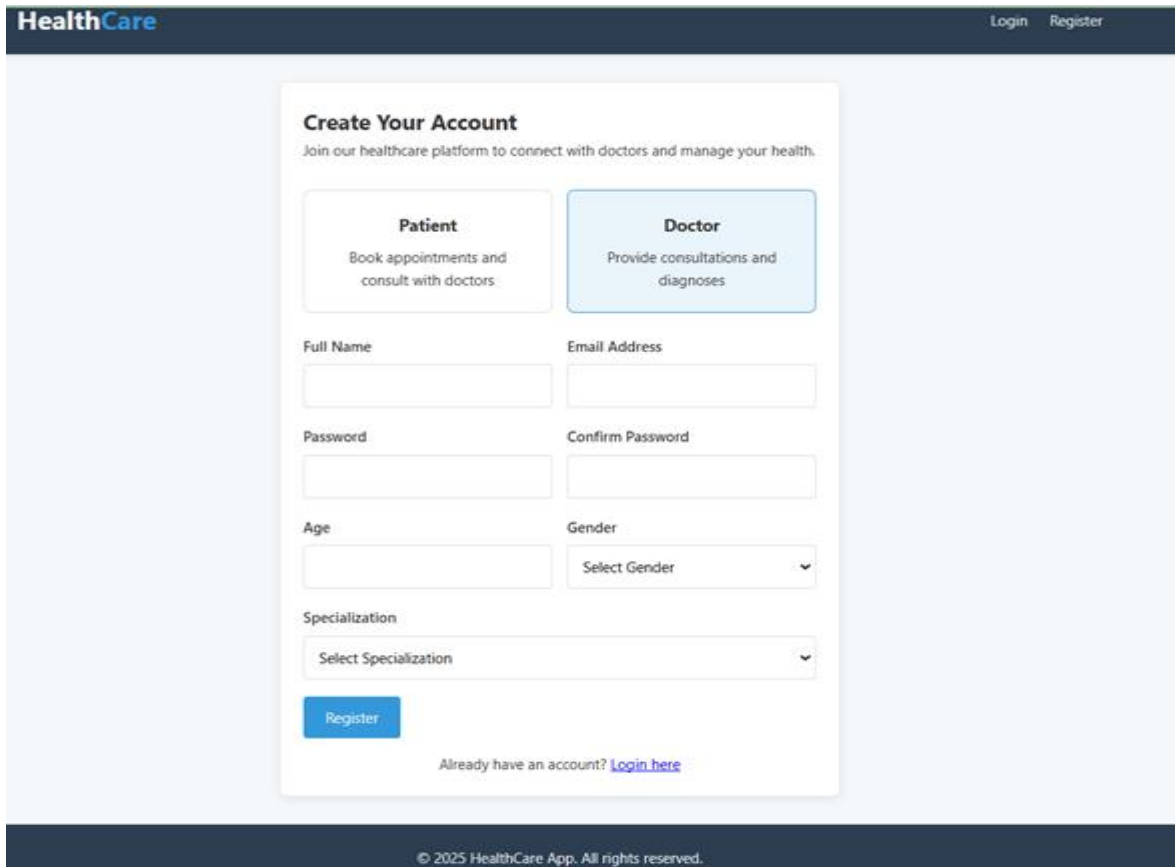
The Home Page serves as the initial interface that directs users to the key functionalities of your web application.



DOCTOR AND PATIENT REGISTRATION PAGE :

The Doctor Registration Page allows doctors to register and create an account on the platform. It typically includes:

- **Input Fields:** For doctor details such as name, specialty, qualifications, and contact information.
- **Login Credentials:** Fields for setting a username and password for secure access.
- **Submit Button:** A button to submit the registration details, which will then be stored in the database after validation.



The image shows a screenshot of the 'Create Your Account' page for the HealthCare App. The page has a dark blue header with the 'HealthCare' logo on the left and 'Login' and 'Register' links on the right. The main content area is light blue and contains a white card with the title 'Create Your Account' and a subtitle 'Join our healthcare platform to connect with doctors and manage your health.' Below the subtitle are two tabs: 'Patient' (with the description 'Book appointments and consult with doctors') and 'Doctor' (with the description 'Provide consultations and diagnoses'). The 'Doctor' tab is currently selected. The form fields are arranged in two columns: 'Full Name' and 'Email Address' in the first row; 'Password' and 'Confirm Password' in the second row; 'Age' and 'Gender' (a dropdown menu with 'Select Gender' and a downward arrow) in the third row; and 'Specialization' (a dropdown menu with 'Select Specialization' and a downward arrow) in the fourth row. A blue 'Register' button is located at the bottom left of the form. Below the button is a link: 'Already have an account? [Login here](#)'. At the very bottom of the page, there is a dark blue footer with the text '© 2025 HealthCare App. All rights reserved.'

PATIENT AND DOCTOR LOGIN PAGES:

The Patient and Doctor Login Pages allow users to securely access their accounts on the platform. Each login page typically includes:

1. Username and Password Fields: Users enter their credentials (username and password) to authenticate their account.
2. Login Button: A button to submit login details and validate user access.

Once logged in, patients and doctors are redirected to their respective dashboards to manage appointments, medical records, and other relevant tasks.

HealthCareLoginRegister

Login to Your Account

Access your healthcare dashboard to manage appointments.

Patient

Doctor

Email Address

Password

[Forgot password?](#)

Login

Don't have an account? [Register here](#)

© 2025 HealthCare App. All rights reserved.

HealthCareLoginRegister

Login to Your Account

Access your healthcare dashboard to manage appointments.

Patient

Doctor

Email Address

Password

[Forgot password?](#)

Login

Don't have an account? [Register here](#)

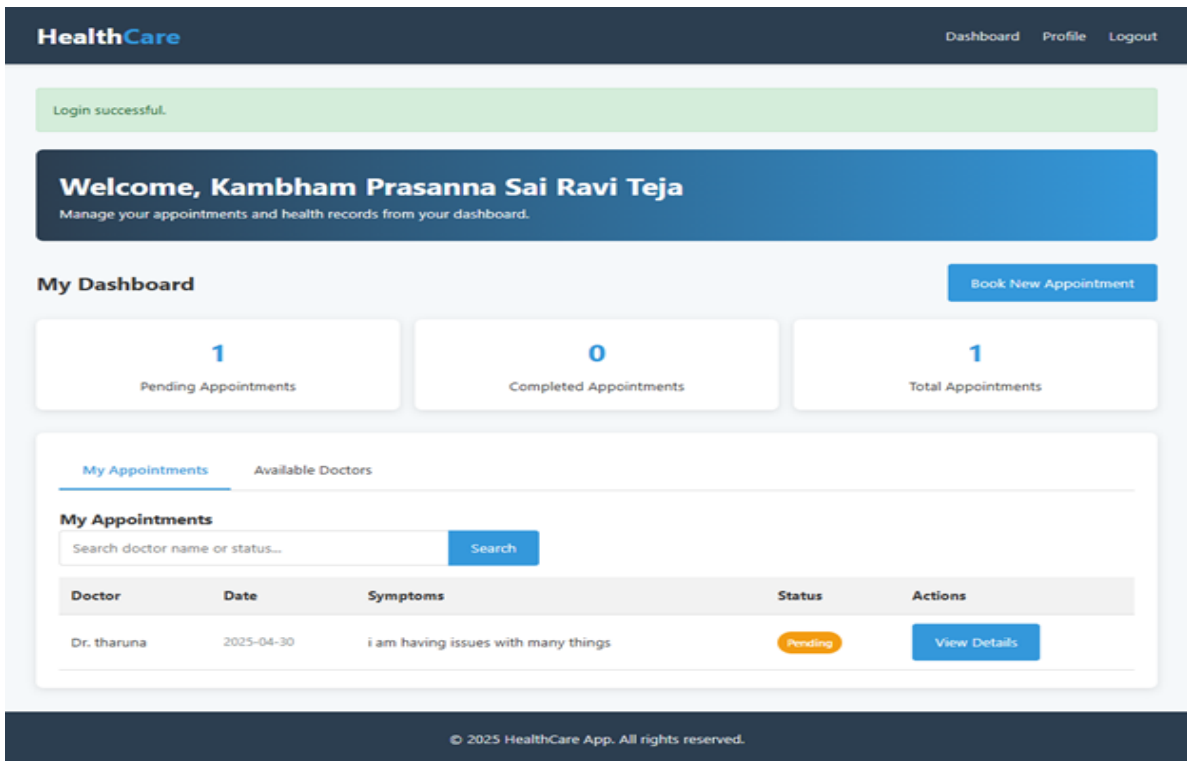
© 2025 HealthCare App. All rights reserved.

User Dashboard:

The User Dashboard (for patients) provides an easy interface to manage appointments and track their status. It typically includes:

1. Book Appointment Section: A form for selecting a doctor, choosing an appointment time, and submitting the request.
2. Appointment Status: A section showing the current status of appointments (e.g., confirmed, pending, or completed) with options to view details or cancel.
3. Upcoming Appointments: A list of future appointments with relevant details such as doctor name, date, and time.

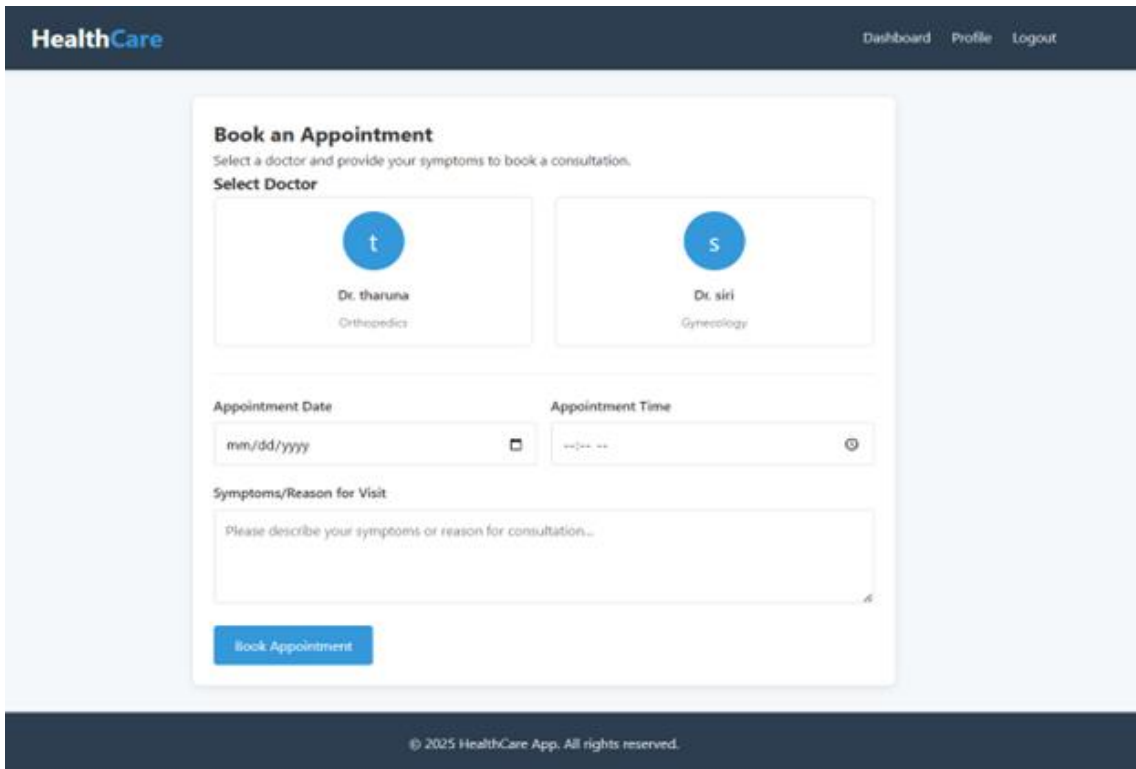
This dashboard helps patients book new appointments and keep track of their healthcare schedules.



The screenshot displays the HealthCare app's user dashboard. At the top, a dark blue header contains the 'HealthCare' logo and navigation links for 'Dashboard', 'Profile', and 'Logout'. Below the header, a green banner indicates 'Login successful.'. A blue welcome banner for 'Kambham Prasanna Sai Ravi Teja' prompts the user to manage appointments and health records. The 'My Dashboard' section features three summary cards: '1 Pending Appointments', '0 Completed Appointments', and '1 Total Appointments'. A 'Book New Appointment' button is located to the right. Below these cards, a section for 'My Appointments' includes a search bar and a table of appointment details.

Doctor	Date	Symptoms	Status	Actions
Dr. tharuna	2025-04-30	i am having issues with many things	Pending	View Details

© 2025 HealthCare App. All rights reserved.



HealthCare Dashboard Profile Logout

Book an Appointment

Select a doctor and provide your symptoms to book a consultation.

Select Doctor



Dr. Tharuna
Orthopedics



Dr. Siri
Gynecology

Appointment Date

mm/dd/yyyy 

Appointment Time

--:--:-- 

Symptoms/Reason for Visit

Please describe your symptoms or reason for consultation...

Book Appointment

© 2025 HealthCare App. All rights reserved.

Doctor Dashboard:

The Doctor Dashboard provides doctors with a comprehensive view of their upcoming appointments and patient details. It typically includes:

1. **Upcoming Appointments List:** A table or list showing patient names, appointment times, and appointment statuses (e.g., confirmed, pending).
2. **Patient Details:** Quick access to each patient's medical history, contact information, and previous visit records.
3. **Appointment Actions:** Options to view, confirm, reschedule, or cancel appointments, ensuring efficient management.

The dashboard serves as the main interface for doctors to manage their schedules, track patient interactions, and provide timely care.

HealthCare

DashboardProfileLogout

Login successful.

Welcome, Dr. tharuna
Manage your appointments and patient consultations from your dashboard.

Doctor Dashboard

Search patient name...Search

1
Pending Appointments

0
Completed Appointments

1
Total Appointments

Pending AppointmentsCompleted AppointmentsAll Appointments

Patient Name	Date	Symptoms	Status	Actions
Kambham Prasanna Sai Ravi Teja	2025-04-30	i am having issues with many things	Pending	View Details

© 2025 HealthCare App. All rights reserved.

HealthCare

DashboardProfileLogout

Appointment Details

Patient Information
Name: Kambham Prasanna Sai Ravi Teja
Status: Pending
Date: 2025-04-30
Created: 2025-04-18T08:09:23.011463

Patient Symptoms
i am having issues with many things

Diagnosis

Enter your diagnosis

Treatment Plan

Describe the treatment plan

Prescription

Write prescription details

Submit Diagnosis

Back to Dashboard

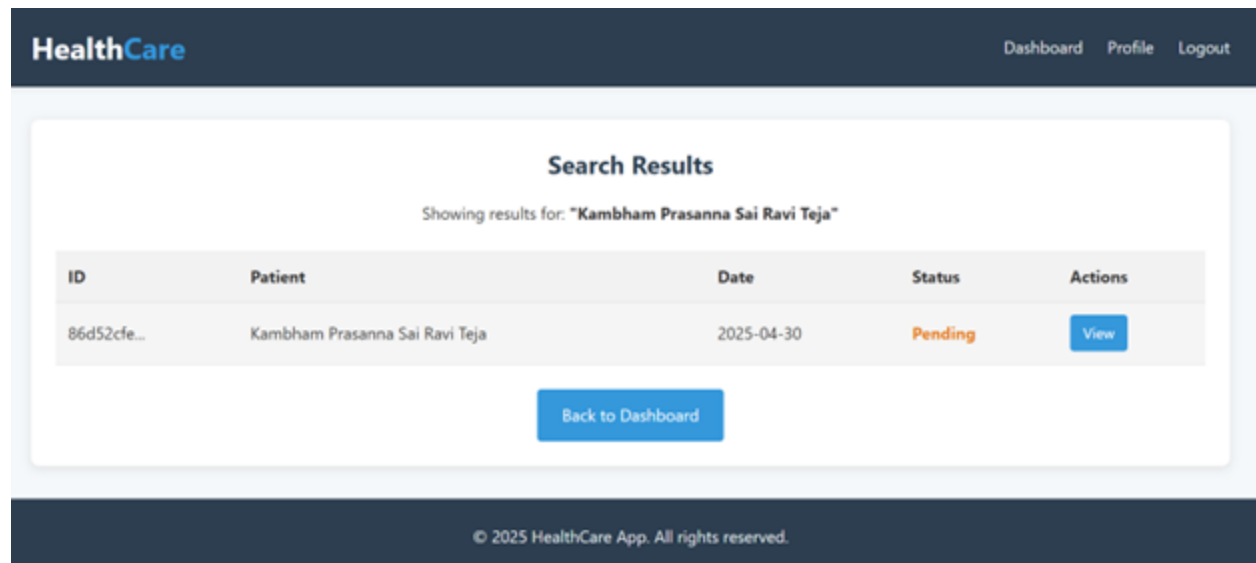
© 2025 HealthCare App. All rights reserved.

Search Appointment:

The **Search Appointment** feature in the **Doctor Dashboard** allows doctors to quickly find and manage specific patient appointments. It typically includes:

1. **Search Bar:** A field where doctors can enter a patient's name, appointment date, or other relevant details to filter appointments.
2. **Appointment List:** Displays the search results, including patient names, appointment times, and statuses, for easy access.
3. **Action Options:** Options to view, update, or cancel appointments directly from the search results.

This feature streamlines appointment management, making it easier for doctors to find and handle patient requests.



The screenshot shows the 'Search Results' page of the HealthCare app. The header includes the 'HealthCare' logo and navigation links for 'Dashboard', 'Profile', and 'Logout'. The main content area displays a table of search results for the query 'Kambham Prasanna Sai Ravi Teja'. The table has columns for ID, Patient, Date, Status, and Actions. A single result is shown with ID '86d52cfe...', Patient 'Kambham Prasanna Sai Ravi Teja', Date '2025-04-30', and Status 'Pending'. An action button labeled 'View' is next to the status. Below the table is a 'Back to Dashboard' button. The footer contains the copyright notice '© 2025 HealthCare App. All rights reserved.'

ID	Patient	Date	Status	Actions
86d52cfe...	Kambham Prasanna Sai Ravi Teja	2025-04-30	Pending	View

Dynamodb Database updations :

Users table :

In the Users Table of DynamoDB, the data structure is designed to store user-related information for both patients and doctors. Typical updates include:

1. **Add New Users:** When a new patient or doctor registers, their details such as name, email, role (patient/doctor), contact info, and password hash are added to the table.
2. **Update User Info:** If a user updates their profile (e.g., changing contact details), the corresponding record in the table is modified.
3. **Status Tracking:** Track the status of user accounts (active, inactive) based on their activity or admin updates.

The Users Table serves as the central repository for all user data, enabling quick access and modification of details when necessary.

Table: UsersTable - Items returned (4) Actions ▼ Create item

Scan started on April 18, 2025, 19:27:34

< 1 > ⚙

☐	email (String) ▼	age ▼	created_at ▼	gender ▼	login_count ▼	name
☐	sunder@thesmartbrid...	25	2025-04-18...	male	2	sunder
☐	gtharunasri19@gmail...	21	2025-04-18...	female	1	tharuna
☐	sairaviteja478@gmail...	21	2025-04-18...	male	2	Kambhan
☐	siri@thesmartbridge.c...	24	2025-04-18...	female	1	siri

2. Appointment table :

In the Appointment Table of DynamoDB, the data structure stores information related to patient appointments. Typical updates include:

1. **Add New Appointment:** When a patient books an appointment, details such as patient ID, doctor ID, appointment date, time, and status (pending, confirmed, canceled) are stored.
2. **Update Appointment Status:** As appointments are confirmed, rescheduled, or canceled, the status field in the table is updated accordingly.
3. **Appointment History:** Historical data about completed appointments can also be stored to track past interactions between patients and doctors.

The Appointment Table allows for efficient management of appointments, ensuring accurate and up-to-date scheduling information for both doctors and patients.

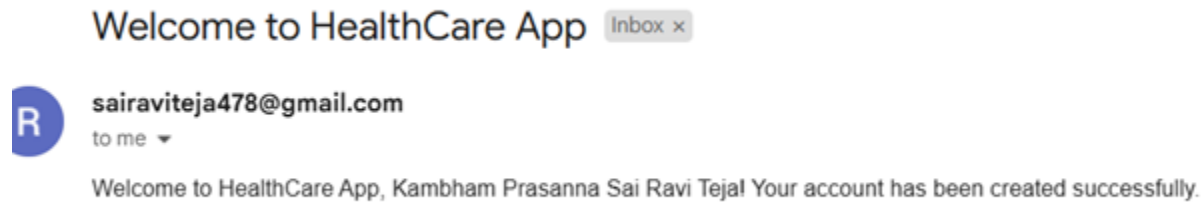
Table: AppointmentsTable - Items returned (2) Actions ▼ Create item

Scan started on April 18, 2025, 19:27:09

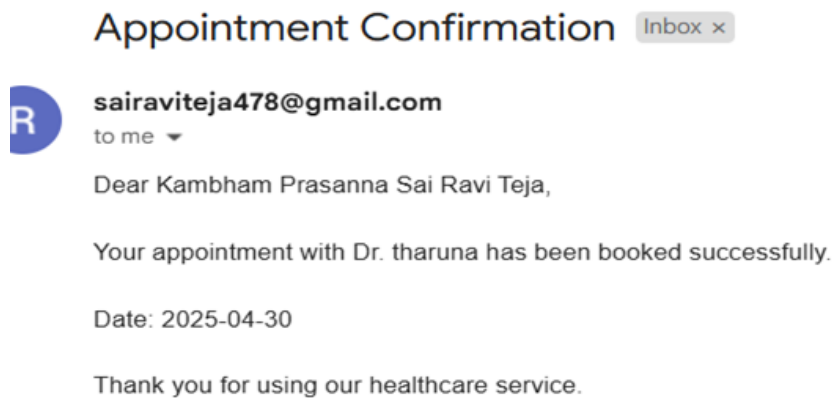
< 1 > ⚙

☐	appointment_id (String) ▼	appointment_date ▼	created_at ▼	diagnosis ▼	doctor_email
☐	86d52cfe-cce6-46b0-941d-1...	2025-04-30	2025-04-18...		gtharunasri19...
☐	42e13fef-fbe9-42d1-9905-f...	2025-04-21	2025-04-18...	make sure t...	siri@thesmart...

1. Mail to the User:



4. Mail to the admin:



CONCLUSION

The MedTrack application has been successfully developed and deployed using a robust cloud-based architecture tailored for modern healthcare environments. Leveraging AWS services such as EC2 for hosting, DynamoDB for secure and scalable patient data management, and SNS for real-time alerts, the platform ensures reliable and efficient access to essential medical tracking services. This system addresses critical challenges in healthcare such as managing patient records, monitoring medication schedules, and ensuring timely communication between healthcare providers and patients.

The cloud-native approach enables seamless scalability, allowing MedTrack to support increasing numbers of users and data without compromising performance or reliability. The integration of Flask with AWS ensures smooth backend operations, including patient registration, medication reminders,

and health updates. Thorough testing has validated that all features—from user onboarding to alert notifications—function reliably and securely.

In conclusion, the MedTrack application delivers a smart, efficient solution for modernizing healthcare management, improving patient care, and streamlining communication between medical staff and patients. This project highlights the transformative power of cloud-based technologies in solving real-world challenges in the healthcare sector.